LEARNLLM • SYSTEMATIC STUDY GUIDE

从零学习 LLM

大语言模型原理、实验与项目实践

原理优先	CPU 可运行	10 个实验	自动测试

从文本、token、注意力与 Transformer 出发，亲手训练 TinyGPT，再理解 SFT、LoRA、评测、RAG 与 Agent。课程按 Windows + CPU 环境设计，无需 GPU、API Key 或预训练模型下载。

参考《Happy-LLM v1.0》知识主线的原创教学改编

LearnLLM v1.0 · 2026-07-15

如何使用本书

这不是一份只供阅读的摘要，而是一门需要运行代码、预测现象、记录失败并用测试验证理解的课程。建议先完整阅读导读，再按 00 到 09 的实验编号推进。每次只改变一个变量。

环境原则

- 所有 Python 命令明确调用当前项目的 .venv，不向系统 Python 安装包。
- 必做实验默认使用 CPU，不需要 API Key、GPU 或外部预训练模型。
- 玩具模型用于验证原理与训练管线，不代表生产级 LLM 能力。

第一条命令

CODE · POWERSHELL

```
.\.venv\Scripts\python.exe .\experiments\00_environment_check.py
```

快速验证

CODE · POWERSHELL

```
.\.venv\Scripts\python.exe -m unittest discover -s .\tests -v
```

改编说明：本书参考用户提供的《Happy-LLM v1.0》章节主线，讲解、代码、数据、测试和练习均为本项目重新设计。附录给出物理页码对照与勘误，避免盲目照抄示例。

目录

如何使用本书 2

环境原则 2

第一条命令 2

快速验证 2

先建立完整心智模型 9

从零学习 LLM：总学习指导 10

1. 学完之后你应该真正会什么 10

2. 先建立一个正确的总图 10

3. 十个阶段的学习路线 11

4. 建议的八周节奏 14

5. 每个实验都按同一套方法做 14

6. 必须能识别的十四个误区 14

7. 结课时如何判断自己真的掌握了 15

版本 2 实验规范 15

实验基础 16

00 学习路线与环境：先建立一间不会污染系统的实验室 17

你最终要学会什么 17

本节问题 17

直觉：先检查实验台，再做实验 18

定义 18

本节的“形状”观念 18

Windows：从零建立并验证 .□venv 19

运行前先预测 . . . 19
预期输出与验收 . . . 20
常见故障与排查 . . . 20
实验记录方法 . . . 21
练习 . . . 21
掌握检查 . . . 22

01 数学与 PyTorch 基础：把“模型猜测”变成可以训练的数 . . . 23

本节问题 . . . 23
直觉：logits 是选票，Softmax 是换算规则 . . . 23
1. 标量、向量、矩阵和张量 . . . 24
2. 点积与矩阵乘法 . . . 24
3. Softmax . . . 25
4. 交叉熵 . . . 25
5. 梯度和一次优化更新 . . . 26
Windows 运行命令 . . . 26
运行前先预测 . . . 27
预期输出与验收 . . . 27
常见故障与排查 . . . 28
练习 . . . 28
掌握检查 . . . 29

02 Tokenizer 与语言模型：从一串文字到“预测下一个 token” . . . 30

本节问题 . . . 30
第一部分：Tokenizer . . . 30
运行 Tokenizer 实验 . . . 32
第二部分：语言模型 . . . 33

运行 Bigram 实验 . . . 35
常见故障与排查 . . . 36
练习 . . . 36
掌握检查 . . . 37
模型结构 . . . 38
03 注意力与 Transformer：让每个位置只读取它应该看到的上下文 . . . 39
本节问题 . . . 39
第一部分：缩放点积注意力 . . . 39
形状推导：单头注意力 . . . 40
因果掩码 . . . 41
运行注意力实验 . . . 42
第二部分：多头注意力 . . . 42
第三部分：位置、残差、归一化与前馈网络 . . . 43
一个 Decoder Transformer Block . . . 44
Encoder、Decoder 与本项目选择 . . . 44
运行 Transformer Block 实验 . . . 45
最重要的无泄漏测试 . . . 46
常见故障与排查 . . . 46
练习 . . . 47
掌握检查 . . . 48
04 模型家族与训练生命周期 . . . 49
本章要回答的问题 . . . 49
1. 从 mask 推导三类架构 . . . 49
2. 三种目标函数 . . . 50
3. LLM 的训练生命周期 . . . 50

4. 一个 loss mask 纸笔实验 . . . 51
5. 常见误区 . . . 51
6. 出门测 . . . 51
训练与适配 . . . 52
05 迷你 GPT：预训练与生成 . . . 53
本章成果 . . . 53
1. 先认识配置 . . . 53
2. 训练前先做三项预测 . . . 53
3. 一步训练逐行解释 . . . 54
4. 训练集与验证集 . . . 55
5. Checkpoint 保存了什么 . . . 55
6. 生成实验 . . . 55
7. 把 base 交给指令微调 . . . 56
8. 单变量探索 . . . 56
9. 常见故障 . . . 57
10. 出门测 . . . 57
版本 2：运行记录、EOS 和配置消融 . . . 57
06 SFT、LoRA 与对齐 . . . 58
一句话先区分 . . . 58
0. 先确认实验关系 . . . 58
1. SFT 的 label mask . . . 59
2. LoRA 原理 . . . 60
3. 为什么 B 初始化为 0 . . . 60
4. 合并权重 . . . 60

5. LoRA 运行后应该看到 . . . 60
6. 怎样读四分支指标 . . . 61
7. rank 与 alpha . . . 62
8. LoRA 与 QLoRA . . . 62
9. 偏好对齐的边界 . . . 62
10. 常见错误 . . . 62
11. 出门测 . . . 63
版本 2：失败记录和扩展评测 . . . 63

应用系统 . . . 64

07 评测、RAG 与 Agent . . . 65

1. 先分清评测对象 . . . 65
2. 困惑度 . . . 65
3. RAG 的完整数据流 . . . 65
4. Tiny RAG 实验 . . . 65
5. Recall@k . . . 66
6. RAG 失败矩阵 . . . 66
7. 文档切块的取舍 . . . 66
8. “有来源”仍不等于“来源支持答案” . . . 67
9. Agent 是系统，不是单个提示词 . . . 67
10. 消融实验 . . . 67
11. 出门测 . . . 68

独立实践 . . . 69

08 结课项目：可复现的微型 LLM 实验报告 . . . 70

任选一个方向 . . . 70
必须提交的目录 . . . 70

报告结构 . . . 71
最低复现命令 . . . 71
评分量表（100 分） . . . 71
完成定义 . . . 71
可复制的版本 2 配置消融 . . . 71
模板、参考与勘误 . . . 73
实验记录模板 . . . 74
基本信息 . . . 74
1. 问题 . . . 74
2. 运行前预测 . . . 74
3. 控制变量 . . . 74
4. 结果 . . . 74
5. 解释 . . . 74
6. 失败与排查 . . . 75
7. 结论边界 . . . 75
8. 出门测 . . . 75
《Happy-LLM v1.0》章节对照、公式索引与勘误 . . . 76
1. 全书物理页对照 . . . 76
2. 课程实验相对前一阶段增加了什么 . . . 77
3. 公式索引：保留原书符号，再说明课程中的含义 . . . 78
4. 不要盲目照抄：原书示例勘误与课程处理 . . . 81
5. 如何使用这份对照 . . . 81

导读

先建立完整心智模型

从概率语言模型到 Transformer、训练、适配和应用系统的全景路线

从零学习 LLM：总学习指导

1. 学完之后你应该真正会什么

“会用聊天机器人”和“理解 LLM”是两件不同的事。本课程把最终能力写成可以检查的行为。完成后，你应该能：

1. 把一句文本完整追踪为 token id、embedding、注意力输出、logits 和下一个 token。
2. 手算小规模 Softmax、交叉熵和缩放点积注意力，并逐行对应到代码。
3. 解释 Encoder-only、Encoder-Decoder、Decoder-only 的信息流和训练目标。
4. 实现带因果掩码的多头自注意力和 Decoder Block，并证明没有未来泄漏。
5. 在 CPU 上训练一个微型 Decoder-only Transformer，保存、加载并生成文本。
6. 区分预训练、SFT、LoRA、量化、偏好对齐；知道它们分别改变数据、目标、参数或数值表示中的哪一部分。
7. 用损失、困惑度、Recall@k、延迟与失败案例评价模型/系统，而不只凭“看起来像人话”。
8. 搭建本地 Tiny RAG，返回检索证据、支持性判定和明确拒答，并能判断问题出在检索还是回答层。
9. 实现带工具白名单、状态轨迹、错误处理和最大步数的本地 Agent 控制循环。

2. 先建立一个正确的总图

Decoder-only LLM 学习的核心概率是：

$$P(x_1, x_2, \ldots, x_T) = \Pi_{t=1}^{T} P(x_t | x_{<t})$$

含义不是“模型从数据库找到唯一答案”，而是：给定已经出现的 token，模型为下一个 token 给出一个条件概率分布。训练时，正确的下一个 token 是监督信号；推理时，程序从这个分布中选择或采样下一个 token，再把它接回输入。

一次前向传播可以画成：

CODE · TEXT

```
token ids [B,T]
   |
   ├── token embedding [B,T,C]
   └── position embedding [T,C]
            | 相加
            ▼
     Transformer Block × L
   ┌──────────────────────────────────────────┐
   | Norm -> causal attention    |
   |       -> residual add      |
   | Norm -> MLP -> residual    |
   └──────────────────────────────────────────┘
            |
            ▼
        logits [B,T,V]
            |
      Cross-Entropy / Sampling
```

符号第一次出现时都要读成中文：

- `B` ：batch size，一批有多少条序列。
- `T` ：sequence length，每条序列有多少个 token。
- `C` ：model dimension，每个位置的隐藏向量维度。
- `H` ：attention heads，注意力头数。
- `D` ：head dimension，通常 `D = C / H` 。
- `V` ：vocabulary size，词表大小。
- `L` ：Transformer Block 层数。

3. 十个阶段的学习路线

阶段 0：环境和实验方法

要回答的问题：怎样确保“我的代码能复现”，而不是“昨天碰巧运行过”？

- 学会明确调用 `. venv\ Scripts\ python. exe` 。
- 理解解释器、包目录、CPU/GPU、随机种子。
- 运行 `experiments/ 00_ environment_ check. py` 。
- 验收：`sys. prefix != sys. base_ prefix` ，路径指向项目 `. venv` 。

阶段 1：最小数学与自动求导

要回答的问题：模型怎样把“猜错了多少”变成每个参数应该往哪边调整？

- 向量点积和矩阵乘法。
- logits、稳定 Softmax、负对数似然、交叉熵。
- 梯度、链式法则、`loss. backward()` 。
- 实验：`01_ math_ foundations. py` 。
- 验收：极大 logits 不溢出；手写交叉熵与 PyTorch 一致。

阶段 2：Token 与统计语言模型

要回答的问题：神经网络为什么不能直接读取字符串？

- 字符、词、子词 Tokenizer 的取舍。
- `encode` 、`decode` 、词表、未知 token、特殊 token。
- Bigram 条件概率、加一平滑、困惑度。
- 实验：`02_ tokenization. py` 、`03_ bigram_ language_ model. py` 。
- 验收：往返编码无损；Bigram 困惑度低于均匀随机基线。

阶段 3：Attention

要回答的问题：一个位置怎样有选择地读取其他位置？

$$\text{Attention}(Q,K,V) = \text{softmax}\leqslant ft((QK\hat{}T)/(\surd(d_k)))V$$

- Q/K/V 是输入经过三个可学习线性投影后的向量；它们不是字面数据库字段。
- `QK^T` 得到每个查询对每个键的分数。

- 除以 `sqrt(d_k)` 避免高维点积让 Softmax 过早饱和。
- 因果 mask 把未来位置的分数变为负无穷，使其 Softmax 权重为 0。
- 实验：`04_attention.py`。
- 验收：每行权重和为 1；上三角未来权重全为 0。

阶段 4：Transformer Decoder

要回答的问题：Attention 为什么还不等于完整 Transformer?

- 多头拆分和拼接。
- token/position embedding。
- 残差连接、Norm、逐位置 MLP。
- 多层堆叠和语言模型输出头。
- 实验：`05_transformer_block.py`。
- 验收：输出 `[B,T,V]`；改变未来 token 不影响过去位置 logits。

阶段 5：模型家族

要回答的问题：BERT、T5、GPT 到底差在哪里?

家族	可见上下文	典型训练目标	典型用途
Encoder-only	双向	MLM	理解、分类、抽取
Encoder-Decoder	Encoder 双向；Decoder 因果	Seq2Seq 去噪/生成	翻译、摘要、条件生成
Decoder-only	因果单向	CLM	通用文本生成、LLM

验收不是背表，而是能为一个新任务画出输入、mask、输出和 loss 位置。

阶段 6：预训练 TinyGPT

要回答的问题：一次标准训练迭代具体发生什么?

CODE • TEXT

```
抽取 [B,T+1] token
  -> x = 前 T 个，y = 后 T 个
  -> logits = model(x)
  -> CE(logits, y)
  -> zero_grad -> backward -> clip -> optimizer.step
  -> 定期 validation / checkpoint
```

- 实验：`06_train_tiny_gpt.py`。
- 先用 `--quick` 检查管线，再增加步数。
- 验收：loss 比初始值明显下降；保存包含配置、权重和冻结 Tokenizer 的真实 base checkpoint；加载后输出一致。
- 后续实验 07 和 10 默认通过 `outputs/pretrain/latest.json` 找到 base，不得在微调阶段重建词表或位置嵌入。版本 2 冻结词表并包含 EOS，预训练保存语料副本与指纹；SFT 独立记录指令数据和评测规则指纹。扩展评测无需重训；改变词表字符或窗口上限则需要重训。
- 重要限制：在小语料上过拟合只证明训练管线工作，不证明模型理解世界。

阶段 7：自回归生成与评测

要回答的问题：同一组模型权重为什么能生成不同风格文本?

- Greedy：总选概率最大 token，稳定但容易重复。
- Temperature：用 `logits / temperature` 改变分布尖锐程度。
- Top-k：只在概率最高的 k 个候选中采样。
- 实验：`07_generate.py`。
- 验收：固定随机种子可复现；能解释采样策略没有增加模型知识。
- 实验 10 生成 adapter 后，实验 07 也可用 `--run-dir <SFT运行目录> --branch lora --instruction "为什么需要因果掩码？"` 在独立进程中严格恢复 LoRA 模型；loader 会先校验 artifact 版本、config、Tokenizer、SHA-256、targets 和 tensor 结构。

阶段 8：SFT、LoRA 和对齐

要回答的问题：预训练之后，怎样让模型按指令回答？怎样少训练参数？

- SFT 改变训练样本格式和 loss mask，通常只对 assistant 回复计算 loss。
- LoRA 冻结原参数，以 `BA` 表示低秩增量：

$$W = W_0 + \Delta W = W_0 + BA$$

- 量化改变参数的数值表示；LoRA 改变可训练参数集合；两者不是同义词。
- 偏好对齐使用 chosen/rejected 或奖励信号改变输出偏好；不是简单的事实注入。
- 实验：先运行 `06_train_tiny_gpt.py` 生成 base，再运行 `10_sft_tiny_gpt.py`；`08_lora.py` 是独立的 Linear 原理演示。
- 实验 10 从同一 base 独立比较 pretrained base、random-init Full SFT、pretrained Full SFT 和 pretrained LoRA-SFT，而不是把 LoRA 接在 Full SFT 后面。各机制的学习率会单独打印并写入产物 metadata；这是起点与评测设置受控的机制对照，不是所有超参数相同的单变量实验。
- 数据按 4/4/4 隔离：train 是四条 `seen`，dev 是四条 `paraphrase`，test 包含两条 `paraphrase` 与两条 `new_intent`。dev 用于选择训练配置；每个分支和 seed 的 test 只在训练完成后评一次，不能反复看 test 再调参。
- 验收 SFT：prompt 与 padding label 都是 `-100`；训练 loss 下降；Full SFT 参数确实改变并保存完整模型权重 checkpoint（不含续训所需的 optimizer 状态）。
- 验收 LoRA：基础权重逐元素不变；只有 A/B 可训练；注入前后输出一致；adapter-only 保存与恢复一致。
- 比较时同时记录 train/dev/test assistant loss、任务成功率、关键词准确率、格式准确率、原预训练语料 validation loss 的保持度/干扰代理、可训练/总参数量、耗时与生成案例。默认 seed 42/43/44；loss 与任务指标跨 seed 报告均值和总体标准差，参数量和耗时按 seed 保留。JSON manifest 保存逐样本、逐类别和运行环境，CSV 保存长格式训练历史。
- 严格任务成功要求一条生成命中全部概念组、未命中声明的矛盾规则并满足全部格式规则。这个 Tiny 字符模型得到 0% 严格成功率也可能是正确、应保留的失败结果；`new_intent` 不代表知识在预训练语料中从未出现，不能宣称未知知识泛化。

阶段 9：RAG、Agent 和系统评测

要回答的问题：模型外部的资料、工具和控制代码如何参与回答？

- RAG：索引 -> 检索 -> 基于证据回答或拒答 -> 返回来源 -> 检查支持性。
- Agent：规划器 + 工具 + 状态 + 有界控制循环，而不是“产生意识”。生产系统可由 LLM 充当规划器，但权限与终止条件仍必须由程序保证。
- 实验：`09_tiny_rag.py`、`11_local_agent.py`。
- 验收 RAG：分别报告 Retrieval Recall、拒答准确率、回答支持率和端到端成功率；资料外问题不得伪造引用。

• 验收 Agent：算术与检索工具走白名单；未知工具、工具异常和最大步数分别进入可观察的停止状态。

4. 建议的八周节奏

周	内容	最低交付物
1	环境、Python、张量、矩阵	实验 00/01 记录；手算一个 Softmax
2	Tokenizer、Bigram、困惑度	实验 02/03；解释 token 不等于词
3	Q/K/V、缩放、mask	实验 04；手画 3×3 causal mask
4	多头、残差、Norm、MLP	实验 05；完成无未来泄漏测试
5	BERT/T5/GPT 与训练生命周期	模型家族对比图；CLM/SFT loss mask
6	TinyGPT 预训练和生成	真实 base checkpoint、冻结 Tokenizer、loss 记录、三种采样结果
7	SFT、LoRA、评测	四分支对比；4/4/4 train/dev/test；三类任务指标与保持度代理；多 seed JSON/CSV；完整模型权重与 LoRA adapter
8	RAG、Agent 与结课项目	检索/拒答/支持指标；Agent 轨迹；消融与失败案例

每天 45-90 分钟即可。每周最后一次不学新概念，只做复述、测试和修复。

5. 每个实验都按同一套方法做

1. 写问题：今天要解释哪个机制?
2. 写预测：运行前预计 shape、数值关系或趋势是什么?
3. 手算极小例子：2-3 个 token 足够。
4. 运行最小代码：先不改参数。
5. 对照预期：不是只看“没有报错”。
6. 单变量实验：只改一个设置。
7. 加自动断言：把理解写成可重复检查。
8. 记录失败：错误信息、根因、修复和仍未解释的点。
9. 用自己的话复述：尽量不看讲义。

6. 必须能识别的十四个误区

1. LLM 不是知识数据库，而是条件概率模型。
2. token 不固定等于一个汉字、一个英文单词或一个语义单位。
3. embedding 相近是训练后产生的表示性质，不是人为给每个维度命名。
4. Q/K/V 是学习投影；字典查询只是直觉比喻。
5. Attention 权重不是模型完整推理过程，也不是可靠解释。
6. 因果 mask 只限制当前前向传播读取未来位置，不会删除模型参数中的知识。
7. 上下文窗口不是长期记忆。
8. 训练 loss 下降不等于理解；验证 loss 上升通常是过拟合信号。
9. 不同 Tokenizer/词表下的困惑度不能直接机械比较。
10. SFT、LoRA、量化、偏好对齐解决的问题不同。

11. assistant-only loss 仍要按“预测下一个 token”右移标签；只把 prompt 原位替换为 `-100` 可能错一位。
12. RAG 通常不修改模型参数，也不能保证答案必然正确；资料不足时拒答是能力而不是失败。
13. Agent 的工具权限、参数校验和最大步数不能只依赖模型自觉遵守。
14. 玩具模型验证的是机制和工程管线，不能外推为生产 LLM 能力。

7. 结课时如何判断自己真的掌握了

不看代码回答以下问题，并用实验验证其中至少三个：

- 为什么稳定 Softmax 要先减最大 logits?
- 为什么语言模型训练的标签相对输入右移一位?
- `QK^T` 的 shape 为什么是 `[B,H,T,T]` ?
- 不除以 `sqrt(d_k)` 可能发生什么?
- 怎样用测试证明模型没有看到未来 token?
- 残差连接和归一化分别解决什么训练问题?
- 为什么初始交叉熵常在 `ln(V)` 附近?
- Temperature 为 0.5 和 1.5 各会怎样改变分布?
- LoRA 的 rank 变大时，可训练参数如何变化?
- assistant-only SFT 中，为什么第一个回答 token 的 label 对齐在最后一个 prompt token 的输入位置?
- 检索正确但回答错误、检索错误但回答看似正确，应分别怎样诊断?
- 怎样用测试证明 Agent 遇到未知工具、工具异常或无限规划时一定会停?

如果某一题只能背定义，就回到相应实验，构造一个最小反例。

版本 2 实验规范

- 旧模型统一重训，执行实验 06→10；不转换、不删除旧 checkpoint。
- 每次实验使用独立运行目录，SFT 保存 base 副本及每个 seed 的 random/full/lora 产物。可整体搬迁，按 manifest 的相对路径和哈希恢复。
- EOS 参与 SFT 监督，生成到 EOS 停止；指令推理使用 `--instruction` ，文本续写使用 `--prompt` 。
- 失败与效果不达标分开记录。默认保留所有 seed；严格验收 `--strict-checks` 保存后返回 2，执行错误返回 1，中断返回 130。
- 演示数据仍为 4/4/4；扩展评测 `--eval-suite extended` 使用 4/20/40。概念同义组和矛盾规则增加可解释性，不代表通用语义正确率。
- 自动输出 PNG/SVG； `scripts/plot_experiments.py` 可从历史 JSON/CSV 重新绘图。不同配置与数据分开标注，不混合汇总。结课项目提供两种配置的可复制命令。

第一部分

实验基础

隔离环境 · 数学与 PyTorch · Tokenizer 与语言模型

00 学习路线与环境：先建立一间不会污染系统的实验室

> PDF 对照：本项目把《Happy-LLM v1.0》物理页 7-14 的 NLP 与文本表示、物理页 15-20 的注意力、物理页 21-24 的 Encoder-Decoder，以及物理页 25-30 的 Transformer 搭建，重排为一条可在本机 CPU 上逐步验证的实验路线。本文的措辞、推导、示例与实验设计均为本项目原创改写；页码仅用于回到 PDF 查阅对应主题。

你最终要学会什么

这条路线不以“调用一次现成大模型接口”为终点，而以回答下面的问题为终点：

1. 一段文字如何变成整数 token?
2. 整数 token 如何变成可训练的向量?
3. 模型如何给“下一个 token”分配概率?
4. 注意力为什么需要 Query、Key、Value 和缩放因子?
5. 因果掩码如何阻止模型在训练时偷看答案?
6. 多头注意力、前馈网络、残差连接和归一化如何组成 Transformer Block?
7. 为什么训练损失下降并不等于模型真正理解了世界?

前六个实验形成一条最小闭环：

阶段	对应实验	核心产物	PDF 物理页
环境	@@INLINE0000@@	可复现且隔离的 Python/PyTorch 环境	7-30 的实验前置
数学	@@INLINE0000@@	张量、矩阵乘法、Softmax、交叉熵	12-13、16-20
Tokenizer	@@INLINE0000@@	文本与 token id 的双向映射	7-13
语言模型	@@INLINE0000@@	最小的下一个 token 概率模型	12-13
注意力	@@INLINE0000@@	缩放点积注意力与因果掩码	15-20
Transformer	@@INLINE0000@@	可前向传播的 Decoder Block	21-30

如果脚本的实际文件名在后续版本中变化，以 `experiments` 目录中的编号 `00` 到 `05` 为准；编号表达的学习顺序不会变化。

本节问题

为什么一定要使用虚拟环境?

因为同一台电脑上的不同项目，可能需要不同版本的 Python 包。直接向系统 Python 安装依赖，会让这些项目互相影响：今天升级 NumPy 解决了 A 项目，明天 B 项目可能就无法运行。

虚拟环境 `. venv` 相当于项目自己的工具箱：

- Python 可执行文件位于当前项目的 `.venv` 中；
- `pip` 安装的包进入 `.venv`，而不是系统环境；
- 删除项目时可以连同 `.venv` 一起删除，不必清理系统 Python；
- 不需要“激活”环境也能使用它，只要始终显式调用 `\.venv\Scripts\python.exe`。

为什么实验要固定随机种子?

神经网络会随机初始化权重，也常会随机抽取训练样本。如果每次运行都完全不同，就很难判断结果变化究竟来自代码修改，还是来自随机性。固定种子不是消除所有误差，而是让小实验更容易复现和比较。

直觉：先检查实验台，再做实验

做化学实验前，会先确认器皿干净、标签正确、仪器校准。LLM 实验也一样：

- `sys.executable` 是正在使用的“仪器”；
- `sys.prefix` 是包的安装位置；
- PyTorch 和 NumPy 版本是实验条件；
- 随机种子检查是一次简单校准；
- CPU/GPU 只是计算设备。这里显示 CPU 完全正常，因为当前课程刻意把模型缩小到 CPU 可运行的规模。

定义

Python 运行时提供两个很有用的路径：

- `sys.base_prefix`：创建虚拟环境所用的基础 Python；
- `sys.prefix`：当前解释器的环境根目录。

在标准 `venv` 中，应满足：

CODE · TEXT

```
sys.prefix != sys.base_prefix
```

同时，当前解释器路径应落在项目内：

CODE · TEXT

```
D:\path\to\LearnLLM\.venv\Scripts\python.exe
```

这两个条件一起使用，比只看命令行前面是否出现 `(.venv)` 更可靠。

本节的“形状”观念

环境检查还没有进入模型，但从现在开始要养成记录形状的习惯。约定：

符号	含义	示例
B	batch size，一次处理多少条样本	2
T	sequence length，一条样本有多少个 token	8
D	embedding/model dimension，每个 token 的向量长度	16
V	vocabulary size，词表中 token 的数量	128
H	number of heads，注意力头数	4

符号	含义	示例
`D_ h`	每个头的维度，`D / H`	`4`

后续看到 `(B, T, D)` 时，不要把它当作三个神秘数字。它只是在说：一批序列、每条序列的若干位置、每个位置的一条向量。

Windows：从零建立并验证 `. venv`

以下命令均在 PowerShell 中执行。先进入项目目录：

```powershell
Set-Location 'D:\path\to\LearnLLM'
```

项目已经提供初始化脚本。即使 PowerShell 不允许直接执行本地脚本，也可以只为这一次进程使用 Bypass：

```powershell
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\bootstrap.ps1
```

初始化完成后，不依赖“激活”动作，直接检查解释器：

```powershell
.\.venv\Scripts\python.exe -c "import sys; print(sys.executable); print('isolated =',
    sys.prefix != sys.base_prefix); print('prefix =', sys.prefix); print('base =',
    sys.base_prefix)"
```

再检查 pip 的归属位置：

```powershell
.\.venv\Scripts\python.exe -m pip --version
```

最后运行第一个实验：

```powershell
.\.venv\Scripts\python.exe .\experiments\00_environment_check.py
```

这个脚本做的每一项检查都在下面，和仓库同一个提交：

 00_environment_check.py <noscript> 看代码需要 JavaScript。源文件在 GitHub：experiments/00_environment_check.py。 </noscript>

注意：以后安装包也始终使用下面的形式，而不是裸写 `pip install . . .` ：

```powershell
.\.venv\Scripts\python.exe -m pip install -r .\requirements.txt
```

运行前先预测

在按 Enter 之前，先把你的预测写下来：

1. `sys. executable` 应该包含哪个目录?
2. `isolated` 应该是 `True` 还是 `False` ?
3. 如果没有 NVIDIA 显卡，实验是否应该失败?
4. 连续两次使用相同种子生成的随机张量，应该相同还是不同?

推荐答案：路径应包含 `.\venv\Scripts`；`isolated` 应为 `True`；没有 GPU 不应失败；同一种子产生的检查值应一致。

预期输出与验收

具体版本号和随机数可能随依赖版本变化，但输出应具有下面这些信息：

```text
Python executable: ...\LearnLLM\.venv\Scripts\python.exe
Virtual environment: True
NumPy: <版本号>
PyTorch: <版本号>
Device: cpu
Reproducible random check: True
Environment check passed.
```

以不变量为准，不要机械比对示例中的每个字符：

- [ ] Python 路径位于当前项目的 `.\venv\Scripts`；
- [ ] 虚拟环境检查为真；
- [ ] NumPy 和 PyTorch 可以导入；
- [ ] 设备为 `cpu` 时仍然通过；
- [ ] 相同随机种子的结果相同；
- [ ] 进程以退出码 `0` 结束。

还可以用下面的命令确认依赖确实安装在虚拟环境中：

```powershell
.\.venv\Scripts\python.exe -c "import numpy, torch; print(numpy.__file__);
    print(torch.__file__)"
```

两个路径都应位于 `.\venv\Lib\site-packages` 下。

常见故障与排查

1. 找不到 `.\.venv\Scripts\python.exe`

现象：PowerShell 报“无法将该项识别为 cmdlet”或路径不存在。

检查：

```powershell
Test-Path .\.venv\Scripts\python.exe
```

若为 `False`，重新执行：

```powershell
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\bootstrap.ps1
```

2. `bootstrap.ps1` 被执行策略阻止

不要修改整台电脑的永久执行策略。使用本节给出的 `powershell -ExecutionPolicy Bypass -File .\...` ，它只影响这次子进程。

3. `ModuleNotFoundError: No module named 'torch'`

通常是依赖没有装入当前 `.\venv`，或者运行了系统 Python。执行：

```powershell
.\.venv\Scripts\python.exe -m pip install -r .\requirements.txt
.\.venv\Scripts\python.exe -c "import torch; print(torch.__version__)"
```

4. 输出显示 `Device: cpu`

这是预期结果，不是降级错误。课程的核心实验不要求 CUDA。将来使用 GPU 时，也必须先让 CPU 小实验通过，因为小实验更快、更容易定位逻辑错误。

5. 中文显示为乱码

先确认终端使用 UTF-8：

```powershell
chcp 65001
$OutputEncoding = [Console]::OutputEncoding = [System.Text.UTF8Encoding]::new()
```

再重新运行实验。源文件和数据文件应保存为 UTF-8。

6. 明明在 `.venv`，`pip` 却指向别处

不要使用裸命令 `pip`。始终使用：

```powershell
.\.venv\Scripts\python.exe -m pip --version
```

`python -m pip` 会把 pip 明确绑定到前面的 Python 解释器。

实验记录方法

每次实验至少记录五项：

```text
日期：
实验编号：
运行前预测：
实际输出：
预测与实际不同的原因：
只改变的一个变量：
```

最重要的习惯是“先预测，再运行；一次只改一个变量”。只看成功输出，容易形成“代码能跑就等于理解”的错觉。

练习

基础练习

1. 用自己的话解释 `sys.prefix` 与 `sys.base_prefix` 的区别。
2. 不激活虚拟环境，运行 `00_environment_check.py`。说明为什么仍能正确使用 `.venv`。
3. 分别运行两次随机数检查，确认固定种子的结果一致。
4. 找出 NumPy 和 PyTorch 的实际安装路径，确认它们不在系统 Python 的 `site-packages` 中。

进阶练习

1. 临时更换实验中的随机种子，只改一个数字。记录输出发生了什么变化、哪些验收条件仍然成立。
2. 画出从文本到 Transformer 输出的形状路线：`文本 -> token ids (B,T) -> embedding (B,T,D) -> logits (B,T,V)`。

3. 解释为什么“固定随机种子”不等于“所有电脑的浮点结果逐位相同”。提示：考虑硬件、线程和数值库。

掌握检查

如果下面任何一项答不上来，先不要急着进入训练：

- [ ] 我能在不激活环境的情况下，明确调用项目 `. venv` 中的 Python。
- [ ] 我能证明 NumPy 和 PyTorch 安装在项目环境，而不是系统环境。
- [ ] 我知道 `cpu` 是本课程支持的正常设备。
- [ ] 我知道随机种子解决的是“便于复现”，不是“保证跨平台逐位相同”。
- [ ] 我能解释 `B`、`T`、`D`、`V`、`H`、`D_h` 的含义。
- [ ] 我会在运行代码前写下预测，并在输出后检查不变量。

完成这些检查后，进入 01 数学与 PyTorch 基础。

01 数学与 PyTorch 基础：把“模型猜测”变成可以训练的数

PDF 对照：《Happy-LLM v1.0》物理页 12-13 从文本表示与语言模型引出“用数表示语言”；物理页 16-20 的注意力公式大量使用点积、矩阵乘法、缩放和 Softmax。本章将这些数学工具提前拆成 CPU 小实验。本文的措辞、推导、示例和代码说明均为本项目原创改写，PDF 页码只用于主题索引。

本节问题

一个 Decoder-only 语言模型在位置 `t` 做的核心事情，可以压缩成一句话：

> 根据位置 `0. . t` 已经出现的 token，给词表中每个“下一个 token”打分，再让正确答案的概率变大。

这句话隐藏了五个问题：

1. 为什么文本最后会变成向量和矩阵?
2. 模型输出的 `logits` 是概率吗?
3. Softmax 为什么能把任意实数变成概率?
4. 为什么直接计算 `exp(1000)` 会出错?
5. 交叉熵怎样把一次预测变成一个可优化的标量 loss?

对应实验，和仓库同一个提交，在这里就能整篇读：

 01_math_foundations.py <noscript> 看代码需要 JavaScript。源文件在 GitHub：experiments/01_math_foundations.py。 </noscript>

直觉：logits 是选票，Softmax 是换算规则

假设词表只有三个 token：

CODE · TEXT

```
["猫", "狗", "。"]
```

模型输出：

CODE · TEXT

```
[2.0, 1.0, -1.0]
```

这三个数称为 logits。它们只是相对分数：`2. 0` 比 `1. 0` 更支持“猫”，但不能说“猫的概率是 200%”。Softmax 像一套换算规则，把所有选票转成非负且总和为 1 的概率。

如果正确答案是“狗”，交叉熵只关心模型分给“狗”的概率。概率越小，惩罚越大；概率越接近 1，惩罚越接近 0。

1. 标量、向量、矩阵和张量

定义

- 标量：一个数，例如 loss `2. 31` 。
- 向量：一列或一行数，例如一个 token 的 embedding，形状为 `(D,)` 。
- 矩阵：二维数表，例如一条序列的 embeddings，形状为 `(T,D)` 。
- 张量：任意维度的规则数组。批量序列通常是三维张量 `(B,T,D)` 。

在 PyTorch 中，它们都由 `torch. Tensor` 表示。

形状

假设：

```text
B = 2  # 两条句子
T = 4  # 每条句子四个 token
D = 8  # 每个 token 用八个数表示
V = 6  # 词表有六个 token
```

典型数据流是：

```text
token ids       (B,T)   = (2,4)
embeddings      (B,T,D) = (2,4,8)
hidden states   (B,T,D) = (2,4,8)
logits          (B,T,V) = (2,4,6)
target ids      (B,T)   = (2,4)
loss            ()      = 标量
```

`()` 表示零维张量，也就是一个标量。

2. 点积与矩阵乘法

定义与公式

两个长度都为 `D` 的向量 `x` 、 `y` 的点积：

$$x \cdot y = \Sigma_{(i=1)}^{(D)} x_i y_i$$

点积把两条向量压成一个数。在注意力中，它被用作 Query 与 Key 的匹配分数。

矩阵乘法把许多次点积并行完成。若：

```text
X: (T,D)
W: (D,V)
```

则：

```text
X @ W: (T,V)
```

可以把它理解为：序列的每个位置都通过同一个线性映射，得到对词表中 `V` 个 token 的分数。

形状检查法

矩阵乘法 `(a,b) @ (b,c) -> (a,c)` 。中间两个 `b` 必须相等，并在结果中消失。

例如：

```text
(B,T,D) @ (D,V) -> (B,T,V)
```

PyTorch 会把前面的 batch 维保留下来。

3. Softmax

定义与公式

对一组 logits `z`：

$$softmax(z_i)=\frac(e^(z_i))(\Σ_j e^(z_j))$$

它保证：

1. 每个概率都大于 0；
2. 所有概率之和为 1；
3. 较大的 logit 对应较大的概率；
4. 所有 logits 同时加上同一个常数，概率不变。

第 4 点非常重要。因为：

$$\frac(e^(z_i-c))(\Σ_j e^(z_j-c))$$
$$=$$
$$\frac(e^(z_i)/e^c)(\Σ_j e^(z_j)/e^c)$$
$$=$$
$$\frac(e^(z_i))(\Σ_j e^(z_j))$$

所以可以选择 `c = max(z)`，先减去最大值再求指数：

$$softmax(z_i)$$
$$=$$
$$\frac(e^(z_i-\max(z)))(\Σ_j e^(z_j-\max(z)))$$

这称为稳定 Softmax。最大指数变成 `exp(0)= 1`，可避免 `exp(1000)` 溢出。

Softmax 沿哪一维?

语言模型 logits 的形状通常是 `(B,T,V)`。我们要在每个 batch、每个时间位置上，对整个词表分配概率，因此沿最后一维 `V` 做 Softmax：

```python
probs = softmax(logits, dim=-1)
```

结果仍是 `(B,T,V)`，并满足：

```text
probs.sum(dim=-1) -> (B,T)，每个元素约等于 1
```

4. 交叉熵

定义与直觉

如果正确 token 的 id 是 `y`，模型给它的概率是 `p_ y`，单个位置的负对数似然为：

$$L=-\log(p_y)$$

几个直观数值：

正确答案概率 `p_y`	loss `-ln(p_y)`	含义
1.0	0	完全确信且答对
0.5	约 0.693	只有一半把握
0.1	约 2.303	给正确答案的概率很低
0.01	约 4.605	非常自信地忽略了正确答案

一批序列的交叉熵，就是对有效位置的 `-log(p_y)` 求平均。实际实现通常直接接收 logits，并在内部用数值稳定的 `log_softmax` ，不要先手工 Softmax 再取 log。

形状展开

语言模型训练时：

```text
logits:  (B,T,V)
targets: (B,T)
```

交叉熵实现常把前两维合并：

```text
logits.reshape(B*T, V): (B*T,V)
targets.reshape(B*T):   (B*T,)
loss:                   ()
```

目标中每个整数都必须落在 `[0, V-1]` 。

5. 梯度和一次优化更新

loss 只是评价一次预测有多差。训练还需要计算：每个参数改变一点，loss 会怎样变化。这就是梯度。

一次最简更新可以写成：

$$\theta \leftarrow \theta - \eta \nabla_\theta L$$

- `θ` ：模型参数；
- `∇θL` ：loss 对参数的梯度；
- `η` ：学习率。

PyTorch 的基本顺序：

```python
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

先清梯度是因为 PyTorch 默认累加梯度。忘记 `zero_grad()` 会让更新混入前几步的梯度。

Windows 运行命令

进入项目根目录：

CODE · POWERSHELL

```
Set-Location 'D:\path\to\LearnLLM'
```

先确认核心包可导入：

CODE · POWERSHELL

```
.\.venv\Scripts\python.exe -c "from learn_llm.math_utils import stable_softmax,
    cross_entropy_from_logits; print('math utilities ready')"
```

运行本章实验：

CODE · POWERSHELL

```
.\.venv\Scripts\python.exe .\experiments\01_math_foundations.py
```

如果想同时运行相关单元测试：

CODE · POWERSHELL

```
.\.venv\Scripts\python.exe -m unittest discover -s .\tests -v
```

运行前先预测

不要先看输出。对下面问题写下答案：

1. `(2,3) @ (3,4)` 的结果是什么形状？
2. logits `[2,1,-1]` 中哪个 token 概率最大？
3. logits `[2,1,-1]` 和 `[1002,1001,999]` 的 Softmax 是否应相同？
4. 对 `[1000,1001]` 直接求指数可能发生什么？稳定 Softmax 呢？
5. 正确 token 概率从 `0.1` 增加到 `0.8`，交叉熵应上升还是下降？

答案：结果为 `(2,4)`；第一个；应相同；直接指数可能溢出而稳定版本应有限；交叉熵下降。

预期输出与验收

数值可能因打印精度略有不同，脚本的输出结构类似：

CODE · TEXT

```
logits shape: (2, 3) = [batch, classes]
probabilities: <两行概率>
row sums: tensor([1., 1.])
manual cross entropy: <数值>
PyTorch cross entropy: <相同或极接近的数值>
absolute error:       <接近 0>
gradient dL/dlogits:  <有限梯度>
PASS: Softmax 稳定，交叉熵和自动求导结果正确。
```

验收不变量：

- [ ] 矩阵乘法输出形状符合 `(a,b) @ (b,c) -> (a,c)`；
- [ ] Softmax 每行之和与 1 的误差小于约 `1e-6`；
- [ ] 大 logits 的 Softmax 结果没有 `NaN` 或 `inf`；
- [ ] 扩展练习：给所有 logits 加同一个常数，概率基本不变；
- [ ] 手工交叉熵和 PyTorch 结果误差小于约 `1e-6`；
- [ ] 扩展练习：提高正确类别的 logit 后，loss 下降。

不要执着于“概率必须打印成某几个固定小数”。浮点数显示位数可能不同，数学不变量才是验收标准。

常见故障与排查

1. `RuntimeError: mat1 and mat2 shapes cannot be multiplied`

把两个输入形状写在纸上。检查左矩阵最后一维是否等于右矩阵倒数第二维：

```text
(..., D) @ (D, ...)
```

不要靠反复添加 `transpose` 碰运气。先说清楚每一维的语义。

2. Softmax 概率和不是 1

最常见原因是 `dim` 选错。对 `(B,T,V)` logits，应沿词表维 `V`，即 `dim=-1`。

调试时打印：

```python
print(logits.shape)
print(probs.sum(dim=-1).shape)
print(probs.sum(dim=-1))
```

3. Softmax 出现 `NaN`

检查是否直接对原 logits 求 `exp`。应先减去该行最大值。还要检查输入本身是否已经包含 `NaN` 或 `inf`。

4. 交叉熵报目标越界

若词表大小为 `V`，目标 id 必须满足：

```text
0 <= target < V
```

打印 `targets.min()`、`targets.max()` 和 `V`。

5. 手工交叉熵与 PyTorch 不一致

依次检查：

1. 两者是否使用自然对数 `ln`；
2. 是否对同一批位置求平均；
3. 是否先错误地把概率四舍五入；
4. 是否把 logits 当成概率；
5. 是否沿正确的词表维做归一化。

6. 第二次反向传播报计算图已释放

默认情况下，`.backward()` 后计算图会释放。下一步训练应重新执行 forward 得到新 loss，而不是对旧 loss 重复反向传播。

练习

基础练习

1. 手算 `[0,0,0]` 的 Softmax。为什么答案一定是均匀分布？
2. 证明给所有 logits 加常数不改变 Softmax。

3. 构造 `(B, T, D) = (2, 5, 8)` 和 `(D, V) = (8, 11)` 的张量，预测并验证结果形状。
4. 分别计算正确答案概率为 `0.9` 与 `0.01` 时的负对数似然。
5. 把正确 token 的 logit 增加 `1.0`，确认 loss 下降；再把错误 token 的 logit增加 `1.0`，观察 loss。

进阶练习

1. 故意实现一个不减最大值的 Softmax，用 `[1000, 1001, 1002]` 观察失败，再解释稳定版本为什么成功。
2. 为 `stable_softmax` 写一个测试：随机 logits 加上 `12345` 后，概率应在浮点误差内保持一致。
3. 不调用 `torch.nn.functional.cross_entropy`，只用 `logsumexp` 和索引实现交叉熵。
4. 假设模型对 `V` 个 token 给出完全均匀的概率，推导单位置交叉熵为什么等于 `ln(V)`。这个结果将在训练 Mini-LLM 时成为“随机基线”。
5. 比较把 batch 中所有位置求平均与只对非 padding 位置求平均的差异。

掌握检查

- [ ] 我能区分 logits、概率与 loss。
- [ ] 我能根据矩阵乘法两端形状推导输出形状。
- [ ] 我能解释为什么 `(B, T, V)` 的 Softmax 沿 `V` 维计算。
- [ ] 我能写出稳定 Softmax，并解释“减最大值”不改变结果。
- [ ] 我能从正确 token 的概率写出 `-ln(p)`。
- [ ] 我知道语言模型交叉熵为什么把 `(B, T, V)` 与 `(B, T)` 变成一个标量。
- [ ] 我知道初始均匀预测的 loss 约为 `ln(V)`。
- [ ] 我能解释 `zero_grad -> backward -> step` 的顺序。

完成后进入 02 Tokenizer 与语言模型，把本章的概率工具放到真正的文本序列上。

02 Tokenizer 与语言模型：从一串文字到“预测下一个 token”

PDF 对照：《Happy-LLM v1.0》物理页 7-11 概览 NLP 任务并讨论中文分词、子词切分等问题；物理页 12-13 转向文本表示、语言模型、Word2Vec 与 ELMo。本章不照抄书中代码，而是用字符 Tokenizer 和 Bigram 模型重建一条最小可运行链路。本文的措辞、例子、推导与实验均为本项目原创改写，页码只作主题索引。

本节问题

上一章中的模型只看到了数字。真实文本怎样进入这些公式?

我们要回答:

1. token 是字、词还是子词?
2. Tokenizer 与模型分别负责什么?
3. `encode` 、 `decode` 和词表是什么关系?
4. 未登录 token（OOV）为什么麻烦?
5. 语言模型到底在估计哪个概率?
6. 最简单的 Bigram 模型能教会我们哪些 LLM 原理?
7. 负对数似然、交叉熵和困惑度之间是什么关系?

对应实验，和仓库同一个提交，在这里就能整篇读:

 02_tokenization.py03_bigram_language_model.py <noscript> 看代码需要 JavaScript。源文件在 GitHub：experiments/02_tokenization.py、experiments/03_bigram_language_model.py。 </noscript>

第一部分：Tokenizer

直觉：Tokenizer 是可逆的编号协议

计算机训练时不能直接把“语言模型”四个字交给矩阵乘法。Tokenizer 先约定一个词表:

CODE · TEXT

```
id 0 -> <unk>
id 1 -> <bos>
id 2 -> <eos>
id 3 -> 模
id 4 -> 型
id 5 -> 语
id 6 -> 言
```

于是：

```text
"语言模型" -> [5, 6, 3, 4]
```

模型只处理右边的整数。生成后，Tokenizer 再把整数还原成文字。

Tokenizer 不是语言模型：

- Tokenizer 决定如何切分、怎样编号；
- 语言模型学习 token 序列的概率；
- Tokenizer 的 id 本身没有大小语义。`id= 100` 不比 `id= 3` “更重要”。

定义

给定词表 `Vocab`，Tokenizer 至少提供：

```text
encode: text -> list[int]
decode: list[int] -> text
vocab_size: int
```

理想的课程级检查是：

```text
decode(encode(text)) == text
```

真实工业 Tokenizer 可能会规范化空格或 Unicode，因此“逐字节完全相等”不是普遍规律；但本项目的字符 Tokenizer 应尽量保持往返一致，方便初学者定位问题。

字符、词与子词

粒度	优点	缺点	本项目用途
字符	实现简单，中文直观，几乎不会遇到整词 OOV	序列较长，英文语义片段被拆散	核心实验
词	序列较短，单位直观	中文需分词；生僻词和新词会 OOV；词表很大	对比思考
子词	在词表大小与序列长度之间折中，可组合新词	训练与解码更复杂	后续 BPE 扩展

PDF 物理页 8 介绍子词切分。本阶段先用字符级方法，是为了把“切分算法的复杂度”与“语言模型的复杂度”分离。等 Bigram 和 Transformer 流程跑通，再换 BPE，观察变化会更清楚。

特殊 token

常见特殊 token：

- `<bos>`：序列开始；
- `<eos>`：序列结束；
- `<pad>`：把不同长度序列补齐；
- `<unk>`：遇到词表外 token 时的回退符号。

它们必须拥有不同 id。是否需要全部使用，取决于实验。字符 Tokenizer 可以仅用语料实际字符，也可以显式加入特殊 token；关键是训练、生成与解码采用同一约定。

形状

一条文本：

```text
"模型"
```

编码为：

```text
[id_模, id_型]        shape: (T,) = (2,)
```

两条等长文本组成 batch：

```text
[[...], [...]]        shape: (B,T)
```

进入 embedding 后：

```text
token ids (B,T) -> embeddings (B,T,D)
```

Embedding 表可以看作一个矩阵：

```text
embedding weight: (V,D)
```

用 token id 查表，得到该 token 的 `D` 维向量。

运行 Tokenizer 实验

在 PowerShell 中：

```powershell
Set-Location 'D:\path\to\LearnLLM'
.\.venv\Scripts\python.exe .\experiments\02_tokenization.py
```

也可以直接检查课程核心 API：

```powershell
.\.venv\Scripts\python.exe -c "from pathlib import Path; from learn_llm.tokenizer
    import CharTokenizer; text=Path('data/tiny_corpus.txt')
    .read_text(encoding='utf-8'); t=CharTokenizer.from_text(text);
    ids=t.encode('语言模型'); print('V=', t.vocab_size, 'ids=', ids, 'decoded=',
    t.decode(ids))"
```

运行前先预测

1. 同一字符每次出现时，id 应相同还是随机变化？
2. `encode` 输出的长度与字符数有什么关系？
3. 把语料中从未出现的字符传给只从该语料建词表的 Tokenizer，可能发生什么？
4. `decode(encode(text))` 在本实验中应得到什么？
5. 词表顺序改变但模型权重不变，模型含义会保持不变吗？

第 5 题答案是否定的。模型参数是围绕某套 id 协议训练的；更换 id 映射等于把输入符号的含义打乱。

预期输出与验收

脚本先做字符往返，再打印 BPE 合并过程；输出模式应类似：

```text
原文: 语言模型预测下一个 token。
token ids: [...]
还原: 语言模型预测下一个 token。
词表大小: <正整数>
merge 01: (...) (出现 ... 次)
...
PASS: 字符级往返无损，BPE 重复合并高频相邻符号。
```

验收：

- [ ] `vocab_size` 等于词表中的唯一 token 数；
- [ ] 相同字符始终映射到相同 id；
- [ ] 所有编码 id 都满足 `0 <= id < vocab_size`；
- [ ] 对语料内示例，`decode(encode(text)) == text`；
- [ ] 扩展练习：未知字符不会被静默映射成一个普通字符；
- [ ] 同样语料重复构建词表时，id 映射稳定。

第二部分：语言模型

直觉：Bigram 是只有一步记忆的语言模型

Bigram 模型只看当前 token `x_t`，预测下一个 token `x_{t+1}`：

$$P(x_{t+1} \mid x_t)$$

例如语料中“语”后面经常出现“言”，那么 `P(言 | 语)` 应高于一个从未跟在“语”后面的字符。

Bigram 明显不够聪明。看到“苹果公司发布”与“我吃了一个苹果”，当前 token 相同时，它无法利用更早上下文区分含义。但它已经包含现代 LLM 的训练骨架：

1. 把文本编码为 token ids；
2. 用前文预测下一个 token；
3. 得到 logits；
4. 用交叉熵评价；
5. 调整参数或统计量；
6. 按概率逐步生成新 token。

Transformer 主要改变的是“怎样利用更长上下文产生 logits”，而不是把训练目标换成完全不同的东西。

从序列构造训练对

若 token 序列为：

```text
[3, 8, 2, 5]
```

输入与目标错开一位：

```text
inputs  = [3, 8, 2]
targets = [8, 2, 5]
```

也就是：

```text
3 -> 8
8 -> 2
2 -> 5
```

这是后续 Transformer 预训练仍会使用的“shifted targets”。

计数模型

建立计数矩阵：

```text
counts: (V,V)
```

`counts[i,j]` 表示 token `i` 后面出现 token `j` 的次数。第 `i` 行归一化后得到条件分布 `P(next= j | current= i)`。

最直接的最大似然估计：

$$P(j| i)=\frac(C_(ij))(Σ_k C_(ik))$$

如果某一行总计数为 0，分母会为 0；如果某个合法转移从未出现，概率会为 0，负对数似然会变成无穷。可用加法平滑：

$$P(j| i)=\frac(C_(ij)+α)(Σ_k(C_(ik)+α))$$

其中 `α > 0`。`α` 越大，分布越接近均匀；越小，越相信观察到的计数。

NLL、平均交叉熵与困惑度

给定真实序列，平均负对数似然：

$$NLL =-(1)/(N)Σ_(t=1)^(N)log P(x_(t+1)| x_t)$$

困惑度：

$$PPL=e^(NLL)$$

直觉上，PPL 可以粗略理解为模型在每一步面对的“有效候选数”。若模型始终在 `V` 个 token 上均匀预测：

$$NLL=\ln(V), PPL=V$$

因此，在训练语料上，一个学到局部规律的 Bigram 模型通常应比均匀基线 `V` 更低。但训练 PPL 低不代表泛化好，更不代表理解语义。

采样生成

从起始 token `x_ 0` 开始：

1. 读取 `P(next | x_ 0)`；
2. 按概率抽取 `x_ 1`；
3. 再读取 `P(next | x_ 1)`；
4. 重复到指定长度或 `<eos>`。

采样含随机性，所以要固定随机种子。生成文本只需展示学到的二元相邻模式，不应期待它写出连贯文章。

形状总览

```text
encoded corpus        (N,)
inputs                (N-1,)
targets               (N-1,)
transition counts     (V,V)
transition probs      (V,V)
one step probs        (V,)
generated ids         (T_generated,)
```

若使用可训练的 embedding 形式，当前 token id 查到的整行 logits 也可写成：

```text
logit table           (V,V)
inputs                (B,T)
selected logits       (B,T,V)
```

运行 Bigram 实验

```powershell
Set-Location 'D:\path\to\LearnLLM'
.\.venv\Scripts\python.exe .\experiments\03_bigram_language_model.py
```

相关单元测试：

```powershell
.\.venv\Scripts\python.exe -m unittest discover -s .\tests -v
```

运行前先预测

1. `inputs` 和 `targets` 的长度是否相同？
2. 概率矩阵的每一行之和应是多少？
3. 没有平滑时，未见转移的 NLL 可能是什么？
4. 均匀模型的 PPL 与词表大小有什么关系？
5. 固定同一种子采样两次，文本是否应一致？
6. Bigram 能否区分“银行利率上升”和“河流两岸”中更远距离的上下文？

预期输出与验收

脚本使用稀疏计数而不是显式打印整个 `(V,V)` 矩阵；输出模式应包含：

```text
词表大小：<V>
在训练语料中，'语' 后最常见字符：[...]
Bigram 训练语料困惑度：<通常小于 V>
均匀随机基线困惑度：<V>
采样文本：<固定长度文本>
PASS：利用一个 token 的上下文，模型优于均匀随机猜测。
```

关键验收：

- [ ] `inputs[1:]` 与 `targets[:-1]` 表达同一段中间序列；
- [ ] 能解释若把稀疏计数展开，`counts` 、 `probs` 会是 `(V,V)` ；
- [ ] 能用公式说明每个上下文条件分布的概率和为 1；
- [ ] 启用平滑后没有 `NaN` 、 `inf` 或 `log(0)` ；
- [ ] 训练语料 PPL 通常低于均匀基线 `V` ；

- [ ] 扩展练习：重置相同随机种子会产生相同样例；
- [ ] 生成 id 全在合法词表范围内，且能被 decode。

常见故障与排查

1. 中文文件读出乱码

显式使用 UTF-8：

CODE · PYTHON

```
text = path.read_text(encoding="utf-8")
```

PowerShell 显示异常时可先执行：

CODE · POWERSHELL

```
chcp 65001
$OutputEncoding = [Console]::OutputEncoding = [System.Text.UTF8Encoding]::new()
```

2. `KeyError` 或未知字符错误

确认待编码文本是否包含训练语料从未出现的字符。课程实现应明确报错或使用 `<unk>`，不要悄悄丢掉字符。若选择扩充词表，模型也必须用新词表重新初始化或训练。

3. decode 后字符顺序不对

检查 `stoi`（token 到 id）和 `itos`（id 到 token）是否真正互为逆映射。不要分别用两种不同排序构建它们。

4. 概率矩阵出现 `NaN`

检查是否有整行计数为 0，并发生 `0/0`。可以只为已出现的上下文建模，或添加 `α > 0` 的平滑。

5. PPL 是无穷大

真实目标的预测概率中出现了 0。检查平滑，并在取 log 前确认概率为正。不要只用一个很小的 epsilon 掩盖上游计数错误。

6. 生成结果不通顺

这通常不是 bug。Bigram 只记住一个 token 的上下文。先检查相邻字符模式、长度、合法 id 和可复现性，不要用文章质量作为这个实验的验收标准。

7. 训练 PPL 很低，所以认为模型理解了文本

训练指标只说明模型适合这批数据。可以保留一段验证文本，比较 train PPL 与 validation PPL；差距较大表示过拟合或分布差异。

练习

Tokenizer 基础练习

1. 分别对中文、英文、带 emoji 的字符串做字符编码，比较 token 数。
2. 打印词表前 20 个映射，说明 id 的排序规则。
3. 给 Tokenizer 增加 `<bos>` 和 `<eos>`，确保两者 id 不同且解码规则清晰。
4. 构造一个语料外字符，记录实现是报错还是映射 `<unk>`，评价两种策略的利弊。

Bigram 基础练习

1. 手工为文本 `ABABA` 写出全部训练对和 `(A,B)` 、 `(B,A)` 计数。
2. 比较 `α = 0.01` 、 `0.1` 、 `1.0` 时同一行概率的变化。
3. 分别从同一个 token 生成 50 个字符，改变种子并比较文本。
4. 计算均匀分布在 `V=10` 、 `V=100` 时的 NLL 与 PPL。

进阶练习

1. 把数据划分成 train/validation，分别计算 PPL，并解释差距。
2. 将 Bigram 扩展为 Trigram：用最近两个 token 预测下一个 token。估算计数表从 `V^2` 增长到多大。
3. 实现 temperature：用 `logits / temperature` 再 Softmax。比较 `0.5` 、 `1.0` 、 `2.0` 的生成多样性。
4. 保持语料不变，比较字符词表与简单按空格分词的词表大小、序列长度和 OOV 行为。
5. 解释为什么“Tokenizer 词表更大”不一定总是更好。提示：考虑 embedding 参数量、序列长度和稀有 token。

掌握检查

- [ ] 我能区分 Tokenizer 与语言模型。
- [ ] 我能解释字符、词、子词三种粒度的权衡。
- [ ] 我能验证 `encode/decode` 的往返关系与 id 范围。
- [ ] 我能从长度为 `N` 的序列构造 `N-1` 个 next-token 训练对。
- [ ] 我能解释 Bigram 估计的是 `P(x_{t+1}|x_t)` 。
- [ ] 我能写出计数归一化与加法平滑公式。
- [ ] 我能从平均 NLL 得到 PPL，并说明均匀基线为何是 `V` 。
- [ ] 我知道 Bigram 的生成不流畅并不自动表示代码错误。
- [ ] 我知道训练 PPL 下降不等于真正理解或泛化。

完成后进入 03 注意力与 Transformer：模型将不再只看前一个 token，而是学习如何从整个可见上下文中聚合信息。

第二部分

模型结构

注意力与 Transformer · 模型家族与训练生命周期

03 注意力与 Transformer：让每个位置只读取它应该看到的上下文

PDF 对照：《Happy-LLM v1.0》物理页 15-20 依次讨论注意力、缩放点积、自注意力、因果掩码与多头注意力；物理页 21-24 讨论 Encoder-Decoder、前馈网络、归一化和残差；物理页 25-30 讨论 embedding、位置编码和完整 Transformer 的组装。本章以 Decoder-only 路线重写为两个 CPU 实验。所有措辞、推导、例子和代码说明均为本项目原创改写，页码只用于定位 PDF 主题。

本节问题

Bigram 只能看前一个 token。现在我们希望当前位置能读取更长的上下文，但不能在训练时偷看未来答案：

1. Query、Key、Value 各自做什么?
2. 为什么相似度用 `QK^T` ，结果是什么形状?
3. 为什么要除以 `sqrt(D_ h)` ?
4. 因果掩码究竟遮住矩阵的哪一半?
5. 多头注意力为什么不等于简单重复同一个头?
6. token 位置信息从哪里来?
7. 残差、归一化、前馈网络怎样与注意力组成一个 Block?
8. 怎样用测试证明模型没有读取未来 token?

对应实验，和仓库同一个提交，在这里就能整篇读：

 04_attention.py05_transformer_block.py <noscript> 看代码需要 JavaScript。源文件在 GitHub：experiments/04_attention.py、experiments/05_transformer_block.py。 </noscript>

第一部分：缩放点积注意力

直觉：检索、匹配、取回

可以把注意力看成一次可微分检索：

- Query：当前位置在寻找什么；
- Key：每个候选位置用什么标签参与匹配；
- Value：匹配到该位置后，实际取回什么内容。

Query 与 Key 的点积产生匹配分数。Softmax 把分数变成权重，再对 Value 加权求和。

“注意力权重”描述这一次信息混合的比例，但它不自动等于完整、可靠的因果解释。不要把某个权重较大直接翻译为“模型之所以回答，是因为这个词”。

定义与公式

缩放点积注意力：

$$\text{Attention}(Q,K,V) = \text{softmax}\leqslant\text{ft}((QK\hat{}\backslash\text{top})/(\surd(D_h))+M)V$$

其中：

- `Q` 、 `K` 的最后一维都是 `D_ h` ；
- `QK^T` 给出每个 Query 对每个 Key 的分数；
- `sqrt(D_ h)` 是缩放因子；
- `M` 是可选掩码；
- Softmax 沿 Key 位置维计算；
- 最后乘 `V` 得到聚合后的内容。

为什么需要缩放?

若 `q_ i` 、 `k_ i` 是大致独立、均值 0、方差 1 的分量，长度 `D_ h` 的点积：

$$q\bullet\ k=\Sigma_(i=1)\hat{}(D_h)q_i\ k_i$$

其波动规模会随 `D_ h` 增大。过大的分数进入 Softmax 后，分布容易过度尖锐：一个位置接近 1，其余接近 0，梯度也可能变得不利于训练。除以 `sqrt(D_ h)` 把分数拉回较稳定的尺度。

这不是为了让结果“更小看起来漂亮”，而是为了控制统计尺度。

形状推导：单头注意力

先忽略 batch：

```text
Q:       (T_q,D_h)
K:       (T_k,D_h)
V:       (T_k,D_v)
K^T:     (D_h,T_k)
scores:  (T_q,T_k)
weights: (T_q,T_k)
output:  (T_q,D_v)
```

矩阵乘法：

```text
(T_q,D_h) @ (D_h,T_k) -> (T_q,T_k)
(T_q,T_k) @ (T_k,D_v) -> (T_q,D_v)
```

自注意力中，Q、K、V 来自同一序列，因此通常 `T_ q = T_ k = T` ：

```text
scores/weights: (T,T)
```

加入 batch 后：

```
Q,K,V:  (B,T,D_h)
scores: (B,T,T)
output: (B,T,D_v)
```

每一行权重都应满足：

$$\Sigma_{j=1}^{T} A_{ij}=1$$

也就是说，每个 Query 对所有可见 Key 的权重之和为 1。

因果掩码

直觉：并行训练，但不泄露答案

训练序列：

```
<bos> 我 喜欢 学习
```

模型需要同时学习：

```
<bos>       -> 我
<bos> 我    -> 喜欢
<bos> 我 喜欢 -> 学习
```

为了利用矩阵并行计算，我们一次输入完整序列；为了不偷看答案，位置 `i` 只能读取 `j <= i` 的 token。

掩码定义

采用加法掩码时：

$$M_{ij}=\begin{cases}0, & j\leqslant i\\ -\infty, & j>i\end{cases}$$

`j > i` 是当前 Query 右侧的未来 Key。加上 `-inf` 后，Softmax 将其概率变成 0。

长度 `T= 4` 时，可见性矩阵是下三角：

```
query 0: [可见, 屏蔽, 屏蔽, 屏蔽]
query 1: [可见, 可见, 屏蔽, 屏蔽]
query 2: [可见, 可见, 可见, 屏蔽]
query 3: [可见, 可见, 可见, 可见]
```

如果代码用布尔 mask，要先确认 `True` 代表“保留”还是“屏蔽”。不同 API 的约定可能相反。

掩码形状

单头可用：

```
mask: (T,T) 或 (1,T,T)
```

多头 batch 通常使用可广播形状：

```text
mask: (1,1,T,T)
```

它会广播到 `(B,H,T,T)` ，同一条因果规则供所有 batch 和 head 使用。

运行注意力实验

PowerShell：

```powershell
Set-Location 'D:\path\to\LearnLLM'
.\.venv\Scripts\python.exe .\experiments\04_attention.py
```

运行前先预测

1. 若 `Q,K,V` 都是 `(2,4,8)` ，单头输出是什么形状？权重是什么形状？
2. 权重最后一维的和应是多少？
3. 因果模式下， `weights[:,0,1:]` 应接近什么？
4. 第三个位置能否读取第一个位置？能否读取第四个位置？
5. 把 `V` 全部设为同一个向量，所有位置的输出会有什么特点？
6. 去掉 `sqrt(D_h)` 后，较大维度的注意力通常会更平坦还是更尖锐？

预期输出与验收

脚本打印完整的 3×3 教学矩阵；输出模式类似：

```text
Q/K/V shape: (1, 1, 3, 2) ...
attention weights (no mask): <3×3 矩阵>
row sums: tensor([[[1., 1., 1.]]])
attention output: <3×2 矩阵>
causal weights: <上三角为 0 的 3×3 矩阵>
scaled mean entropy: ...
unscaled mean entropy: ...
PASS: 权重逐行归一化，因果掩码完全阻断未来位置。
```

不变量：

- [ ] 输出的 Query 长度与 `Q` 相同；
- [ ] 权重形状最后两维是 `(T_q,T_k)` ；
- [ ] 每一行可见位置的概率和约为 1；
- [ ] 因果模式中所有 `j > i` 的权重接近 0；
- [ ] 输出等于权重对 `V` 的加权和；
- [ ] 输出、权重都没有 `NaN` 或 `inf` ；
- [ ] 关闭因果模式时，未来位置允许获得非零权重。

第二部分：多头注意力

直觉

单头只能在一个投影空间中计算相似度。多头注意力为不同头使用不同的可学习投影，使它们能够形成不同的匹配方式。不能预先规定某个头一定学习语法、另一个头一定学习指代；这是训练可能出现的行为，不是结构保证。

定义

对输入 `X` ：

$$Q=XW_Q,\ K=XW_K,\ V=XW_V$$

将模型维度 `D` 分成 `H` 个头：

$$D_h=D/H$$

每个头独立计算：

$$head_h=Attention(Q_h,K_h,V_h)$$

拼接并投影：

$$MHA(X)=Concat(head_1,\ldots,head_H)W_O$$

必须满足 `D % H = = 0` 。

多头形状追踪

CODE · TEXT

```
input X                    (B,T,D)
Q/K/V projection           (B,T,D)
reshape                    (B,T,H,D_h)
transpose for attention    (B,H,T,D_h)
scores                     (B,H,T,T)
weights                    (B,H,T,T)
per-head output            (B,H,T,D_h)
transpose back             (B,T,H,D_h)
concatenate                (B,T,D)
output projection          (B,T,D)
```

例： `B= 2,T= 5,D= 16,H= 4` ，则 `D_ h= 4` ，注意力分数形状为 `(2,4,5,5)` 。

第三部分：位置、残差、归一化与前馈网络

为什么需要位置信息?

不额外提供位置时，自注意力主要根据内容相似度混合信息，本身没有稳定的绝对或相对顺序标记。Transformer 会加入或注入位置信息。

PDF 物理页 26-28 介绍经典正弦/余弦位置编码。一种常见形式为：

$$PE(pos,2i)=\sin⩽ft(pos/10000\^(2i/D))$$

$$PE(pos,2i+1)=\cos⩽ft(pos/10000\^(2i/D))$$

它的形状是 `(T,D)` ，可广播并加到 `(B,T,D)` 的 token embeddings 上。现代 Decoder-only 模型也常使用 RoPE 等相对位置方案；本阶段先掌握“内容表示必须与位置信息结合”这一结构要求。

残差连接

残差连接：

$$y=x+f(x)$$

它为信息与梯度提供直接路径。相加要求两个张量形状相同，所以注意力子层与 FFN 子层都通常保持最后维度 `D` 不变。

归一化

LayerNorm 对每个 token 的特征维进行归一化。输入和输出形状相同：

```text
(B, T, D) -> (B, T, D)
```

本项目使用的具体归一化以实现为准。关键检查是：归一化不改变 batch、序列长度和模型维度，并保持输出有限。

前馈网络

FFN 对每个位置独立应用同一组非线性变换：

$$FFN(x)=W_2\ \sigma(W_1x+b_1)+b_2$$

形状通常是：

```text
(B, T, D)
 -> Linear: (B, T, D_ff)
 -> activation: (B, T, D_ff)
 -> Linear: (B, T, D)
```

注意力在位置之间混合信息；FFN 在每个位置的特征维上进行更丰富的非线性变换。

一个 Decoder Transformer Block

常见的 Pre-Norm 结构可写为：

$$x_1=x+MHA(Norm_1(x))$$

$$y=x_1+FFN(Norm_2(x_1))$$

数据流：

```text
x (B, T, D)
  -> Norm -> causal MHA -> (B, T, D) -> residual add
  -> Norm -> FFN        -> (B, T, D) -> residual add
y (B, T, D)
```

Block 不负责直接输出词表概率。完整语言模型还需要：

```text
token ids (B, T)
 -> token embedding + position information (B, T, D)
 -> N x Transformer Block (B, T, D)
 -> final norm (B, T, D)
 -> language-model head (B, T, V)
```

Encoder、Decoder 与本项目选择

PDF 物理页 21-24 介绍 Encoder-Decoder：

- Encoder 的自注意力通常可以双向读取整条输入；
- Decoder 的自注意力需要因果掩码；

• Encoder-Decoder 模型还使用 cross-attention，让 Decoder 的 Query 读取 Encoder 产生的 Key/Value。

本项目先实现 Decoder-only Block，因为 next-token 生成只需要一条因果序列：

CODE · TEXT

```
过去 token -> 因果 Decoder -> 下一个 token logits
```

这不是说 Encoder 或 Encoder-Decoder 不重要，而是先用最短路径理解现代生成式 LLM 的核心训练目标。

运行 Transformer Block 实验

CODE · POWERSHELL

```
Set-Location 'D:\path\to\LearnLLM'
.\.venv\Scripts\python.exe .\experiments\05_transformer_block.py
```

相关测试：

CODE · POWERSHELL

```
.\.venv\Scripts\python.exe -m unittest discover -s .\tests -v
```

运行前先预测

设 `B= 2,T= 6,D= 16,H= 4,D_ ff= 64` ：

1. `D_ h` 是多少?
2. 每头 scores 是什么形状？全部头合在一起呢?
3. Block 输入与输出形状是否相同?
4. FFN 中间张量是什么形状?
5. 将第 5 个 token 改掉，位置 0-4 的输出是否应变化？位置 5 呢?
6. 若 `D= 15,H= 4` ，模型应正常运行还是尽早报错?

答案： `D_ h= 4` ；单头 `(2,6,6)` 、合并头 `(2,4,6,6)` ；相同；中间为 `(2,6,64)` ；因果且关闭 dropout 时位置 0-4 不应变化、位置 5 可以变化；应尽早因不可整除而报错。

预期输出与验收

脚本直接从整个 TinyGPT 的 logits 检查因果性；输出模式类似：

CODE · TEXT

```
input shape:  (1, 6) = [B, T]
logits shape: (1, 6, 32) = [B, T, vocab]
parameters:  <参数量>
未来 token 改变后，过去 logits 最大差异: 0.000e+00
PASS: 模型形状正确，并且没有未来信息泄漏。
```

验收：

- [ ] `D % H = = 0` ，并正确计算 `D_ h` ；
- [ ] 拆头、换轴、合头后，元素数量保持一致；
- [ ] Block 输入输出都是 `(B,T,D)` ；
- [ ] FFN 中间维扩展后再回到 `D` ；
- [ ] 注意力和 FFN 两条残差相加的形状完全一致；
- [ ] 扩展练习：forward 输出全为有限数；
- [ ] 扩展练习：对输出求标量 loss 后，参数梯度存在且有限；
- [ ] `eval()` 模式下修改未来 token，不改变过去位置输出；

- [ ] 修改过去 token 可以影响其后位置，说明上下文路径确实存在。

最重要的无泄漏测试

只检查 mask 图案还不够。更强的行为测试是：

1. 创建两条完全相同的 token 序列；
2. 只修改第二条序列最后一个 token；
3. 在 `model.eval()` 与 `torch.no_grad()` 下分别前向传播；
4. 比较最后位置之前的输出或 logits。

如果模型是严格因果的，前缀输出应在浮点误差内相同：

```text
allclose(output_a[:, :-1], output_b[:, :-1]) == True
```

最后位置可以不同，因为它能看到被修改的 token。若前缀也变化，可能存在：

- mask 三角方向反了；
- mask 没有广播到全部 head；
- 位置或 batch 维换错；
- 测试仍在 train 模式，dropout 引入随机差异；
- 比较的并不是同一模型参数下的两次 forward。

常见故障与排查

1. `D` 不能被 `H` 整除

`D_h=D/H` 必须是整数。初始化时立即断言：

```text
D % H == 0
```

不要在 reshape 时报一个难懂的元素数量错误后才发现。

2. scores 的形状不是 `(B, H, T, T)`

检查 `Q, K` 是否先变为 `(B, H, T, D_h)`，并只转置 `K` 的最后两维：

```text
Q @ K.transpose(-2, -1)
```

不要误把 batch 或 head 维转走。

3. 因果 mask 方向反了

手工打印 `T=4` 的 mask。第 0 行只能看到第 0 列；最后一行应能看到全部列。若相反，三角方向或布尔语义错误。

4. 第一行注意力全是 `NaN`

这通常表示第一行所有位置都被屏蔽，Softmax 正在处理一整行 `-inf`。因果 self-attention 必须允许每个位置读取自己，即保留对角线。

5. `view` 报张量不连续

`transpose` 后内存布局可能不连续。可以使用 `reshape`，或在 `view` 前调用 `contiguous()`。但先确认换轴逻辑正确，不要只为消除报错而随意 contiguous。

6. 合并多头后 token 顺序混乱

正确路线是：

```
(B, H, T, D_h) -> transpose -> (B, T, H, D_h) -> reshape -> (B, T, D)
```

不能直接把 `(B, H, T, D_ h)` reshape 成 `(B, T, D)`，否则 head 与 token 的内存次序会混合。

7. 无泄漏测试偶尔失败

先调用 `model. eval()` 关闭 dropout，并在同一个模型上比较。再检查绝对/相对容差，而不是要求浮点逐位相等。

8. 梯度为 `None`

确认参数参与了 forward，loss 由输出计算，且中间没有 `. detach()` 、转换成 NumPy 或包在 `torch. no_ grad()` 中。

9. 梯度出现 `NaN` 或特别大

逐层检查第一个非有限值；确认注意力有缩放、mask 不产生全 `-inf` 行、Softmax 使用稳定实现。不要先用梯度裁剪掩盖前向传播错误。

练习

注意力基础练习

1. 手工设置一个 Query，使它与某个 Key 完全相同、与其他 Key 正交，预测注意力最大位置。
2. 令所有 logits 相同，推导未屏蔽位置上的注意力分布。
3. 对 `T= 5` 写出因果可见性矩阵，数出一共有多少个可见元素。
4. 比较 `causal= False` 与 `causal= True` 的权重矩阵，说明哪些元素必须变化。
5. 把 Value 的每一行设为对应位置编号，解释输出为何是“被关注位置编号的加权平均”。

多头与 Block 练习

1. 对 `D= 24`，列出 `H= 1,2,3,4,6,8` 时的 `D_ h`。
2. 在保持 `D` 不变时改变头数，比较参数量是否一定按头数线性增长。提示：考虑组合的 Q/K/V 投影矩阵。
3. 暂时去掉输出投影 `W_ O`，记录形状并说明表达能力上少了什么混合步骤。
4. 分别画出 Pre-Norm 与 Post-Norm 的数据流，不要求立刻断言哪一个“永远更好”。
5. 将 FFN 扩展倍率从 `4D` 改为 `2D`，计算参数量变化。

关键挑战

1. 自己实现并通过“未来 token 修改不影响过去输出”的行为测试。
2. 去掉因果 mask，证明无泄漏测试会失败；再恢复 mask，证明测试通过。
3. 去掉缩放因子，比较不同 `D_ h` 下注意力权重熵。记录现象，不只描述结论。

4. 画出正弦位置编码前四个通道随位置变化的曲线，观察不同频率。
5. 堆叠两个 Block，验证形状保持 `(B, T, D)` ，并检查所有层都获得梯度。
6. 估算 self-attention 的 scores 张量元素数 `B*H*T*T` 。当 `T` 翻倍时，元素数变为多少倍？这解释了长上下文的主要成本之一。

掌握检查

- [ ] 我能用“检索、匹配、取回”解释 Q、K、V，同时知道这只是直觉类比。
- [ ] 我能从 `Q` 、 `K` 形状推导 `QK^T` 的形状。
- [ ] 我能写出缩放点积注意力完整公式。
- [ ] 我能解释缩放因子为什么是 `sqrt(D_h)` 量级。
- [ ] 我能画出因果 mask，并确认第 0 行只看自己、最后一行看全部过去。
- [ ] 我能追踪 `(B, T, D)` 拆成 `(B, H, T, D_h)` 再合回去的每一步。
- [ ] 我能区分注意力负责“位置间混合”、FFN 负责“每个位置的特征变换”。
- [ ] 我能解释残差相加为什么要求形状不变。
- [ ] 我知道位置编码解决什么问题。
- [ ] 我能用行为测试证明没有未来信息泄漏，而不只目测 mask。
- [ ] 我能说明 Encoder 双向注意力、Decoder 因果注意力和 cross-attention 的差别。

完成本章后，你已经从 token id 走到了一个可运行的 Decoder Transformer Block。下一阶段只需加入数据切片、语言模型头、优化循环与生成策略，就能组成一个真正可训练的 Mini-LLM。

04 模型家族与训练生命周期

对应 PDF：第 3 章（物理页 32-57）、第 4 章（物理页 58-73）。本章不要求下载 BERT、T5 或 GPT 权重；目标是能从“可见信息、输出形式、训练目标”推导它们的差别。

本章要回答的问题

1. Encoder-only、Encoder-Decoder、Decoder-only 为什么不是简单的模型大小区别?
2. MLM、Seq2Seq、CLM 分别让模型预测什么?
3. Pretrain、SFT、偏好对齐各改变了哪一层问题?
4. 为什么本项目先训练 TinyGPT，而不是复刻完整 RLHF?

1. 从 mask 推导三类架构

Encoder-only

一个位置可以读取左右两侧上下文，因此得到的是双向表示。BERT 的典型预训练任务 MLM 会遮掉部分 token，再根据两侧上下文恢复它。

CODE · TEXT

```
输入：  今 天 [MASK] 很 好
可见：  左右两侧都可见
输出：  被遮位置的 token 分布
```

它擅长分类、匹配、实体抽取等“理解型”任务。它不是不能生成任何文字，而是架构和训练目标没有天然形成逐 token 自回归生成接口。

Encoder-Decoder

Encoder 双向读取源序列；Decoder 用因果 mask 读取已经生成的目标，并通过 cross-attention 读取 Encoder 表示。

CODE · TEXT

```
源文本 -> Encoder states ──────────┐
目标前缀 -> causal Decoder <───────┘ -> 下一个目标 token
```

它很适合翻译、摘要等“输入序列到输出序列”的条件生成。T5 进一步把多种任务统一为 text-to-text。

Decoder-only

整个序列都用因果 mask。GPT、LLaMA 以及本项目 TinyGPT 都属于这一类。

CODE · TEXT

```
x1 -> 预测 x2
x1 x2 -> 预测 x3
x1 x2 x3 -> 预测 x4
```

统一的 next-token 接口使自然语言任务很容易表示为“给定提示，继续生成”。

2. 三种目标函数

MLM：Masked Language Modeling

只在被遮位置计算 loss。它使用双向上下文，但预训练时存在人为的 `[MASK]` 形式。

Seq2Seq

给定源序列，让 Decoder 预测目标序列。训练时目标输入通常右移一位；不能把完整目标原样同时作为可见输入和答案。

CLM：Causal Language Modeling

$$\mathcal{L}_{CLM}=-\sum_{t=1}^{T}\log P(x_t \mid x_{<t})$$

实现时从同一 token 窗口构造：

CODE · PYTHON

```
window = token_ids[start : start + T + 1]
x = window[:-1]
y = window[1:]
```

预测问题：若忘记右移，模型可能学到什么投机路径？答案：如果当前位置能直接看到同一标签，就不再是 next-token 学习。

3. LLM 的训练生命周期

PDF 物理页 62 用 Pretrain -> SFT -> RLHF 展示常见流程。更准确地说，这是目标逐步改变，而不是简单“继续多训练几次”。

阶段	数据	目标	得到的能力/倾向
Pretrain	大量通用文本	下一个 token	语言模式、通用表示和知识统计
SFT	指令-回答、多轮对话	在回答位置做 CLM	按任务格式回答、遵循指令
Preference alignment	chosen/rejected 或奖励	偏好目标、KL 约束等	更倾向人类偏好的行为

SFT 的关键不是加一句“请回答”

假设输入是：

CODE · TEXT

```
用户：为什么需要因果掩码？
助手：它阻止当前位置读取未来 token。
```

常见做法是把用户和格式 token 的 label 设为 `-100`，只在助手回答位置计算交叉熵。这样模型接收完整提示，但监督信号集中在要学的回答上。

偏好对齐的最小直觉

同一个 prompt 有回答 A 和 B，人类标注 `A > B`。训练目标让 chosen 的相对分数更高。完整 PPO 会涉及 Actor、Reference、Reward、Critic，工程门槛远超原理入门；DPO 等方法用成对偏好数据构造更直接的目标。

本项目在必做路线中只要求理解数据对、相对偏好和“不要偏离参考模型过多”的原因，不在普通 CPU 上复刻生产 RLHF。

4. 一个 loss mask 纸笔实验

为下面序列标出是否计算 loss：

CODE · TEXT

```
<user> 1+1 等于几？ <assistant> 2 <eos>
```

推荐答案：

CODE · TEXT

```
token: <user> 1 + 1 ... <assistant> 2 <eos>
label:  -100 -100 ...     -100    2 <eos>
```

再思考：多轮对话时，是只监督最后一轮 assistant，还是监督每一轮 assistant？两者都可以，但必须明确数据和目标，不能硬编码某个 Tokenizer 的 token id。

5. 常见误区

- “参数超过某个固定数就自动出现智能”：能力随数据、训练、评测和规模共同变化，所谓涌现也受指标离散化影响。
- “SFT 把所有事实写进模型”：SFT 主要学习任务格式与行为，也会改变知识，但不是可靠数据库写入。
- “RLHF 让回答必然正确”：它优化偏好，可能改善有用性与安全性，但不等于事实证明。
- “Decoder-only 只能看左边，所以完全不理解整句”：在预测较后 token 时，它已经读取了此前整段；限制的是单次位置的信息方向。
- `C≈6ND` 或某个 token/参数比例是永久定律：它们是特定假设下的经验近似，不是跨时代常数。

6. 出门测

1. 解释题：为什么 BERT 的双向注意力不能直接用于标准左到右生成？
2. 计算题：长度 6 的 CLM 序列能形成多少个 next-token 监督位置？
3. 调试题：SFT loss 几乎为 0，但输出只复制 prompt。检查输入/标签是否错位，是否把 prompt 自己作为同位置答案。
4. 设计题：把摘要任务分别表示为 Encoder-Decoder 和 Decoder-only 输入，画出 mask。

掌握标准：不看表格，也能从任务需求画出可见信息和 loss 位置，再判断适合的架构。

第三部分

训练与适配

TinyGPT 预训练与生成 · SFT、LoRA 与偏好对齐

05 迷你 GPT：预训练与生成

对应 PDF：物理页 74-116。原资料直接进入 LLaMA2 组件和较大数据，本项目把配置缩到约几十万参数，并使用仓库内置语料，使你能在 CPU 上观察完整训练闭环。

本章成果

- 训练 `TinyGPT`，看到训练 loss 下降。
- 理解 x/y 右移、batch、反向传播、梯度裁剪和 checkpoint。
- 得到实验 10 共用的真实预训练 base，并理解为什么模型配置与 Tokenizer 必须随 checkpoint 冻结。
- 用 greedy、temperature、top-k 生成文本。
- 解释玩具模型为何会“像语料”，却没有通用语言能力。

1. 先认识配置

默认教学配置：

参数	值	意义
`block_size`	不超过 48	模型一次可读的最长 token 数
`n_layer`	2	Decoder Block 数
`n_head`	4	注意力头数
`n_embd`	64	每个位置的隐藏维度
`vocab_size`	由本地预训练语料决定	输出分类数量；语料前 90% 覆盖后续 SFT train/dev/test 所需字符
`dropout`	0	小实验先消除随机影响

参数量远小于真实 LLM。缩小不会改变 causal attention、残差、MLP、交叉熵和生成循环的逻辑。

2. 训练前先做三项预测

1. 初始模型接近均匀猜测时，loss 大约是 `ln(vocab_size)`。
2. 训练 loss 应下降；验证 loss 不保证一直下降。
3. 小语料上的生成会重复、背诵和断句异常。

把预测写进实验记录，再运行：

CODE • POWERSHELL

```
# 约 40 步，只验证管线
.\.venv\Scripts\python.exe .\experiments\06_train_tiny_gpt.py --quick

# 默认 200 步，观察更清楚的趋势
.\.venv\Scripts\python.exe .\experiments\06_train_tiny_gpt.py
```

训练脚本和第 6 节的生成脚本全文，和仓库同一个提交：

 <span

class="nb-code-tab" aria-selected="true">06_train_tiny_gpt.py07_generate.py <noscript> 看代码需要 JavaScript。源文件在 GitHub：experiments/06_train_tiny_gpt.py、experiments/07_generate.py。 </noscript>

预期输出模式：

```text
参数量: ...
均匀随机理论 loss ln(V): ...
初始 train/val loss: ... / ...
step    1 | train ... | val ...
...
PASS: loss 明显下降，checkpoint 与训练指标已保存。
```

具体浮点数会因 PyTorch 版本和硬件略有差异；趋势和断言才是验收标准。

这一步不是可跳过的“热身”。它会生成 `outputs/pretrain/<run-id>/base.pt`，实验 07 和实验 10 都从这里恢复同一套模型配置、预训练权重与字符 Tokenizer。若还没有该文件，实验 10 会要求先运行实验 06，而不会悄悄退回随机模型。

3. 一步训练逐行解释

```python
x, y = sample_language_model_batch(...)
optimizer.zero_grad(set_to_none=True)
logits, loss = model(x, y)
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
```

`x, y`

从连续 token 中取长度 `T+1` 的窗口。`x` 是前 `T` 个，`y` 是后 `T` 个，所以 `y[t]` 正是 `x[t]` 的下一个 token。

`zero_grad`

PyTorch 默认累加梯度。不清零就会把多个 step 的梯度叠在一起；梯度累积是可用技术，但必须有意控制。

`forward + loss`

模型输出 `[B,T,V]`，每个位置有 V 个 logits。交叉熵选择目标 token 的负对数概率并求平均。

`backward`

自动求导沿计算图计算每个可训练参数的梯度。

`clip_grad_norm_`

当总梯度范数过大时按比例缩小，降低训练突然发散的风险。它不是修复所有训练问题的万能开关。

`optimizer.step`

AdamW 根据梯度、动量统计和学习率更新参数。只有到这里权重才真正改变。

4. 训练集与验证集

本项目把语料前 90% 用于训练、后 10% 用于验证。

- train loss 下降：模型越来越适合见过的训练窗口。
- val loss 下降：对留出的同分布文本也有改善。
- train 降而 val 升：过拟合信号。

由于语料非常小且不是随机打散的独立样本，验证数值只能教学使用，不能当作严谨模型排行榜。

5. Checkpoint 保存了什么

`outputs/pretrain/<run-id>/base.pt` 包含：

- 模型参数 `model_state`。
- 优化器状态用于检查；本课程尚未提供训练恢复命令。
- 当前 step。
- 模型配置。
- 字符 Tokenizer 状态，包括独立 EOS token。
- 随机种子、运行目录内语料副本的相对路径和 SHA-256；评测数据及规则指纹由 SFT run 独立记录。

只保存权重却不保存配置和 Tokenizer，往往无法正确重建模型。Tokenizer id 映射一旦变化，即使 tensor shape 相同，语义也完全错位。

本项目在实验 06 生成词表后就把 Tokenizer 冻结。实验 10 验证 train/dev/test 的字符和长度约束，使用 base 运行目录里的语料副本核对 SHA-256。修改扩展评测无需重新预训练；若加入词表之外的字符或超过 base 上下文上限，则必须重新准备语料或配置并训练，不能在微调阶段偷偷修改模型。预训练与 SFT 各自保存数据快照，确保结果能追溯到实际输入。

实验 07 和 10 使用严格 artifact loader：除了模型 state dict，它还校验格式版本、config、冻结 Tokenizer、数据指纹、tensor 键与形状。加载 LoRA 时还会验证 base SHA-256、targets、rank/alpha/dropout 和 adapter tensor；校验失败就停止，而不是部分加载一个表面上能运行的模型。

位置嵌入同样属于 checkpoint。当前 SFT 序列右移后的最大长度必须不超过 base 的 `block_size`；实验 10 会按 checkpoint 中的上限检查，不会为了容纳更长样本创建另一套位置嵌入。

6. 生成实验

先训练，再运行：

```powershell
.\.venv\Scripts\python.exe .\experiments\07_generate.py --prompt "语言模型" --tokens
    80
```

不传 artifact 参数时，上述命令通过 `outputs/pretrain/latest.json` 加载最近成功的默认目录预训练。可先在新目录训练 adapter，再在独立进程中按 manifest 恢复：

```powershell
$SftRun = ".\outputs\sft\generate-$(Get-Date -Format yyyyMMdd-HHmmssfff)"
.\.venv\Scripts\python.exe .\experiments\10_sft_tiny_gpt.py --quick --output-dir
    $SftRun
.\.venv\Scripts\python.exe .\experiments\07_generate.py `
  --run-dir $SftRun --branch lora `
  --instruction "为什么需要因果掩码？" --tokens 64
```

`--base-checkpoint` 与 `--adapter` 必须成对出现，也不能和 `--checkpoint` 混用。adapter 不是完整模型文件；它只能与元数据所绑定的 base 一起恢复。

脚本比较三种策略：

Greedy

$$x_{t+1}=\arg\max_i p_i$$

优点是确定；缺点是容易陷入高概率重复，且无法探索次高概率的合理延续。

Temperature

$$p_i=\mathrm{softmax}(z_i/\tau)$$

- $\tau < 1$ ：分布更尖锐，更保守。
- $\tau > 1$ ：分布更平，更随机。
- Temperature 不修改模型权重，也不让事实更可靠。

Top-k

只保留 k 个最大 logits，再归一化采样。它减少长尾低概率 token 造成的离题，但 k 太小会损失多样性。

7. 把 base 交给指令微调

完成实验 06 后，可运行同一 checkpoint 上的对照实验：

CODE • POWERSHELL

```
.\.venv\Scripts\python.exe .\experiments\10_sft_tiny_gpt.py --quick
```

实验 10 不会把 Full SFT 的结果再作为 LoRA 的起点。两条预训练微调分支都从同一个 `base.pt` 独立复制，因此 loss、原语料保持度代理、参数量和耗时才可比较。指令数据使用 4/4/4 的 train/dev/test 隔离：dev 用于调参，test 在训练结束后才评一次。详细的四分支设计、任务指标、JSON/CSV 记录与 adapter 文件见下一章。

8. 单变量探索

依次做，不要同时改：

1. 固定 `top_k=20` ，比较 temperature 0.5、1.0、1.5。
2. 固定 temperature 1.0，比较 top-k 1、5、30。
3. 固定所有参数，只换随机种子。
4. 把 `block_size` 从 48 改 16，重新训练，观察局部长距离模式。
5. 增大 `n_embd` ，记录参数量和每步耗时，而不假设一定改善验证 loss。

9. 常见故障

现象	优先检查
loss 不降	y 是否右移；优化器是否包含参数；学习率；是否调用 `step()`
loss 变 NaN	学习率过大；输入/梯度是否有限；是否忘记缩放/裁剪
生成提示 OOV	提示字符是否出现在训练语料；Tokenizer 是否同 checkpoint
加载时报 shape mismatch	配置与 checkpoint 是否一致
实验 10 找不到 base	先运行实验 06；不要让微调脚本随机创建替代 checkpoint
指令字符不在词表	更新预训练语料前 90% 的自然桥接文本，并重新运行实验 06
SFT 序列超过窗口	缩短样本，或用更大 `block_ size` 重新预训练；不能只扩大微调模型
输出乱码/重复	小语料和字符模型的预期局限；采样是否过热/过冷
训练快但验证差	过拟合、切分偏差、语料太小

10. 出门测

1. 为什么 `logits[: , -1, : ]` 可以用于生成下一个 token?
2. `V= 100` 时均匀预测的交叉熵约是多少?
3. 为什么保存模型时必须一并保存 Tokenizer?
4. 为什么“训练集 loss 很低”不能证明模型理解语言?
5. 为什么实验 10 必须复用实验 06 的 Tokenizer 和 `block_ size` ?

版本 2：运行记录、EOS 和配置消融

每次运行在独立目录保存 `pretraining. json` 、`pretraining. csv` 、`base. pt` 、`corpus. txt` 与 `plots/` 。旧版 checkpoint 被明确拒绝，请重跑实验 06→10；不会删除旧文件。默认 `block_ size` 至少为 48 ，且能容纳演示集的完整答案与 EOS，目前为 49。

`--block-size` 、`--n-layer` 、`--n-head` 、`--n-embd` 、`--dropout` 、`--batch-size` 、`--learning-rate` 、`--corpus` 、`--device` 和 `--output-dir` 均可配置。维度必须能被头数整除，训练和验证语料必须长于窗口。显式短窗口适合消融，但后续 SFT 可能因样本过长而拒绝运行。

EOS 是“答案已结束”的独立 token。SFT 把它作为最后一个监督目标；生成达到 EOS 就停止，不必一直输出到 `--tokens` 上限。批次中已结束的行用 EOS 填充，显示文本时跳过特殊 token。

`status` 描述执行结果，`quality_ status` 描述效果。loss 下降不足仍保存完整记录；只有加上 `--strict-checks` 才会在保存后返回退出码 2。运行异常返回 1，中断返回 130，保留已经完成的训练步；不承诺强制杀进程后的完整恢复。图表从 CSV 重建，绘图失败独立记录，不覆盖训练结果。

06 SFT、LoRA 与对齐

对应 PDF：SFT 物理页 66-69、130-134；LoRA 物理页 135-142；偏好对齐物理页 69-72。

一句话先区分

- SFT：改变训练数据与监督位置，让模型学习指令-回答行为。
- LoRA：改变哪些参数参与训练，用低秩增量降低成本。
- 量化：用更低位宽表示权重/激活，降低内存或计算成本。
- 偏好对齐：改变优化目标，让 chosen 相对 rejected 更受偏好。

它们可以组合，但不是同义词。

0. 先确认实验关系

实验 10 是真实的 checkpoint-based 微调对比，必须先有实验 06 生成的 base：

```
.\.venv\Scripts\python.exe .\experiments\06_train_tiny_gpt.py --quick
$SftRun = ".\outputs\sft\lesson-$(Get-Date -Format yyyyMMdd-HHmmssfff)"
.\.venv\Scripts\python.exe .\experiments\10_sft_tiny_gpt.py --quick --output-dir
    $SftRun
```

实验 10 和第 2 节的实验 08 全文，和仓库同一个提交：

 10_sft_tiny_gpt.py08_lora.py <noscript> 看代码需要 JavaScript。源文件在 GitHub：experiments/10_sft_tiny_gpt.py、experiments/08_lora.py。 </noscript>

`--quick` 把每个训练分支固定为 30 步，但仍默认运行 seed 42/43/44。所有关键输入和输出都可配置；例如：

```
.\.venv\Scripts\python.exe .\experiments\10_sft_tiny_gpt.py `
  --steps 120 --seeds 7 11 19 `
  --train-data .\data\tiny_instructions.jsonl `
  --dev-data .\data\tiny_instructions_dev.jsonl `
  --test-data .\data\tiny_instructions_test.jsonl `
  --output-dir .\outputs\sft\my-comparison
```

`--help` 还列出 base/full/adapter 路径、三种学习率、LoRA rank/alpha/dropout、生成长度、train batch size 和设备。自定义数据必须保留相应的 `split` 字段，并满足冻结 Tokenizer 与 base `block_size` 的约束。

实验 10 默认经 `outputs/pretrain/latest.json` 找到 `base.pt` 并恢复并冻结同一套 Tokenizer 与模型配置，然后从这个 base 独立分叉比较：

分支	起点	更新内容	主要用途
pretrained base	实验 06 checkpoint	不训练	微调前基线

分支	起点	更新内容	主要用途
random-init Full SFT	相同 config 与 Tokenizer、随机权重	全部模型参数	观察“只靠四条指令从头训练”
pretrained Full SFT	同一预训练 base	全部模型参数	观察完整微调与原语料干扰迹象
pretrained LoRA-SFT	同一预训练 base	Attention LoRA A/B	观察参数效率与冻结约束

Full SFT 和 LoRA-SFT 是从同一 base 出发的两种微调方式，不是先做 Full SFT、再给它套 LoRA。这样才能公平比较预训练、初始化和可训练参数集合各自的影响。

这里的“公平”指 config、Tokenizer、训练数据、评测数据以及两个预训练分支的起点一致。为了让 30 步 CPU 快速实验都能产生可观察信号，默认学习率是 Full SFT `3e-3` 、random-init/LoRA-SFT `1e-2` ；脚本会明确打印并写入产物 metadata。它不是“所有超参数完全相同”的单变量消融，不能仅凭最终 loss 或耗时断言某种算法普遍更优。

实验 08 则是独立的原理演示：它用一个小型 Linear 展示 LoRA 的初始化、梯度、冻结与权重合并，不读取 TinyGPT checkpoint，也不产出实际适配器。真实 TinyGPT LoRA-SFT 和 adapter-only 保存都在实验 10 中完成。

1. SFT 的 label mask

实验读取结构化 JSONL，把每条样本格式化为 prompt 与 assistant 回复。每行都有稳定 `id` 、 `split` 、 `intent_ family` 、 `evaluation_ category` 、必要关键词和格式规则。Decoder-only 模型要预测“下一个 token”，所以标签不仅要 mask，还要右移对齐：

CODE · PYTHON

```
response_ids = response_ids + [tokenizer.eos_id]
sequence = prompt_ids + response_ids
input_ids = sequence[:-1]
labels = [-100] * (len(prompt_ids) - 1) + response_ids
```

`input_ ids` 与 `labels` 等长。最后一个 prompt token 负责预测第一个 response token，因此第一个有效 label 位于它的输入位置。EOS 是最后一个有效 label，生成时用它决定停止。交叉熵设置 `ignore_ index= -100` 后，prompt 仍是模型可见上下文，但 prompt 目标与右侧 padding 都不贡献监督 loss。

为什么不能硬编码“assistant 开始标记”的 token id？因为换一个 Tokenizer 后，标记可能被切成不同数量和不同编号的 token。应动态编码模板或在构造数据时明确边界。

实验 10 会检查所有序列都不超过 base checkpoint 的 `block_ size` ，并使用 checkpoint 中的冻结 Tokenizer；它不会依据 SFT 数据重建词表。Full SFT 的完整模型权重与 metadata 保存到 `outputs/ sft/ <run-id>/ seed-<seed>/ full/ model. pt` ，但不含 optimizer 状态，不能直接据此无缝续训。

1.1 Train/dev/test 协议

文件	数量	类别	用途
`data/ tiny_ instructions. jsonl`	4	`seen`	唯一参与梯度更新的数据
`data/ tiny_ instructions_ dev. jsonl`	4	`paraphrase`	调步数、学习率、rank 等配置
`data/ tiny_ instructions_ test. jsonl`	4	2 条 `paraphrase` + 2 条 `new_ intent`	配置确定后的一次最终评测

train 与 dev 共享四个意图族，dev 改写提问；test 的两条 `new_ intent` 使用未在 train/dev 出现的意图族。实验会拒绝跨切分重复 id。每个训练分支和 seed 都先完成训练，才在同一个最终评测阶段对 test 运行一次 teacher-forced loss 与 greedy 任务评测。学习者只能根据 dev 调参，不能反复查看 test 结果再选择配置。

这里的 `new_ intent` 只表示“意图族未出现在 SFT train/dev”，不表示相关字符、概念或知识从未出现在预训练语料。它可用于观察有限的意图迁移失败，却不能支持“模型学会未知知识”或“具有独立分布泛化能力”的结论。

2. LoRA 原理

执行：

CODE · POWERSHELL

```
.\.venv\Scripts\python.exe .\experiments\08_lora.py
```

原线性层：

$$h=W_0x$$

LoRA 冻结 `W0`，学习低秩增量：

$$h=W_0x+(\alpha)/(r)BAx$$

若 `W0` 形状为 `[d_ out,d_ in]`：

- `A` 形状 `[r,d_ in]`。
- `B` 形状 `[d_ out,r]`。
- 可训练参数量 `r(d_ in+d_ out)`，而完整权重是 `d_ in*d_ out`。

当 `r` 远小于输入/输出维度时，可训练参数显著减少。

3. 为什么 B 初始化为 0

本项目 A 随机初始化、B 零初始化。因此训练开始时 `BA= 0`，LoRA 层输出与原层完全一致，不会一包裹就破坏预训练函数。

第一步时，B 可以先获得梯度；A 的梯度可能因 B=0 而为 0。B 更新后，后续 A 也能获得梯度。这是正常现象，不要用“第一步 A 梯度必须非零”作为错误断言。

4. 合并权重

推理前可以计算：

$$W_(merged)=W_0+(\alpha)/(r)BA$$

这样部署时仍是一层普通 Linear，不必额外执行两次低秩乘法。本项目实验验证合并前后输出在浮点误差内一致。

实验 10 的真实 LoRA-SFT 默认把每个 Transformer Block 中的 `attention. qkv` 和 `attention. output` 包装为 LoRA，使用 `rank= 4`、`alpha= 8`。其余 base 参数全部冻结；与 token embedding 共享权重的 `lm_ head` 不作为 LoRA target，以免破坏权重绑定。

5. LoRA 运行后应该看到

运行实验 08 时，预期看到单层原理验证：

CODE · TEXT

```
初始 LoRA 增量最大值: 0...
loss: 较大值 -> 很小值
base.requires_grad: False
A/B gradient present: True True
PASS: 原权重冻结，低秩参数完成适配，合并前后输出一致。
```

验收要看四件事：

1. 包裹前后初始输出一致。
2. base 权重训练前后逐元素相同。
3. A/B 是可训练参数并最终有梯度。
4. 合并权重后的输出和 LoRA 分支输出一致。

运行实验 10 时，还要看到 LoRA 注入前后的 TinyGPT logits 一致、所有冻结参数训练前后逐元素不变，以及 adapter-only 文件恢复后的 logits 与训练完成模型完全一致。

6. 怎样读四分支指标

实验 10 对 train/dev/test 都记录 assistant-only loss 和 greedy 生成，并计算三类透明指标：

- `task_success_rate`：一条回答必须命中全部概念组、未命中声明的矛盾规则并满足全部格式规则才记 1，否则记 0；这是最严格的主指标。
- `keyword_accuracy`：逐条计算已命中必要关键词的比例，再对样本取平均。
- `format_accuracy`：是否满足句末、最大句数和禁止前缀等全部规则，再对样本取平均。

JSON 中既保留每条 prediction、命中关键词和分数，也按 `seen`、`paraphrase`、`new_intent` 类别汇总。规则是确定性的教学启发式，不是语义等价或人工质量评审。Tiny 字符模型生成不稳定，严格任务成功率为 0% 完全可能是诚实结果；不要改宽规则或偷看 test 只为得到非零数字。

还要一起阅读：

- `corpus_validation_loss`：回到原预训练语料后 10% 的固定种子采样窗口计算，作为保持度/干扰代理；它只能提示遗忘迹象，不能单独证明灾难性遗忘。
- `trainable_parameters`、`total_parameters` 与 `duration_seconds`：区分 Full SFT 和 LoRA-SFT 的参数与时间成本。
- train/dev/test 生成案例：用于定位失败，不能替代量化指标。

默认运行 seed 42/43/44。JSON 对每个可汇总 loss 和任务指标报告 `mean` 与 population `std`（总体标准差），并保留逐 seed 原始结果；参数量和耗时也按 seed 保留。随机初始化 SFT 即使把 train loss 压低，也不代表学到了通用语言能力。预训练 Full SFT 可能使原语料验证 loss 上升；LoRA-SFT 参数更少，也不保证 dev/test 更好。实验的目标是测出差异，而不是预设某一分支必胜。

每次运行会生成四类产物：

- `outputs/sft/<run-id>/sft_comparison.json`：带 schema 版本的 manifest，包含 CLI 配置、git/运行环境、数据与 artifact SHA-256、逐 seed/分支/样本结果、跨 seed 汇总、不变量和限制。
- 同目录 `sft_comparison.csv`：长格式逐 step 训练 loss 与各最终 loss，便于画曲线或导入表格。
- `outputs/sft/<run-id>/seed-<seed>/full/model.pt`：每个 seed 的预训练 Full SFT 完整模型权重与 metadata（相对于 adapter-only；不含 optimizer 状态）。
- `outputs/sft/<run-id>/seed-<seed>/lora/adapter.pt`：每个 seed 的 LoRA A/B 与 metadata，包含 base/data SHA-256、Tokenizer、config、targets、rank、alpha、dropout 和最终指标。

每个 seed 的 random 分支也保存到 `seed-<seed>/random/model.pt`。run 根目录携带独立 base 与语料副本、数据快照和 SHA-256；整个目录可搬迁。可选 `--full-checkpoint`、`--adapter-output` 仅额外导出首 seed，已有目标拒绝覆盖。非空 run 目录也拒绝复用。

6.1 严格恢复与独立推理

公共 loader 会在构造可用模型前校验 artifact 格式版本、config、Tokenizer、data/base SHA-256、state 键/形状/类型与有限值；LoRA 还会核对 targets、rank、alpha 和 dropout。任何一项不一致都会报错，不会静默 `strict= False` 或部分加载。

adapter 不是完整模型。实验 10 在保存后已经用严格 loader 独立恢复一次并要求 logits 完全一致；也可以直接让实验 07 在新进程中复现这条路径：

CODE · POWERSHELL

```
.\.venv\Scripts\python.exe .\experiments\07_generate.py `
  --run-dir $SftRun --branch lora --seed 42 `
  --instruction "LoRA 为什么节省参数？" --tokens 64
```

`--base-checkpoint` 和 `--adapter` 必须成对传入，且不能与完整模型的 `--checkpoint` 同时使用。

7. rank 与 alpha

- rank `r` ：低秩通道容量；越大，可训练参数越多，表达能力通常更强，但不是必然更好。
- alpha：更新缩放；实现常用 `alpha/ r` ，使不同 rank 的更新尺度更易控制。
- dropout：只作用于 LoRA 分支，是正则化手段。

单变量练习：把 rank 改为 1、2、4，记录参数量、最终 loss 和收敛步数。不要同时改学习率。

8. LoRA 与 QLoRA

QLoRA 通常是“量化冻结的基础模型 + 可训练 LoRA 适配器”。

- 量化减少基础权重内存。
- LoRA 减少需要更新和保存的参数。
- 量化误差、计算内核和设备兼容性是额外变量。

本项目必做实验不引入量化库，因为低秩原理可以在普通 Linear 上清楚验证。

9. 偏好对齐的边界

假设有 `(prompt, chosen, rejected)` ：

CODE · TEXT

```
prompt: 请解释因果掩码
chosen: 给出定义、用途和限制
rejected: 只说“它让模型更聪明”
```

偏好目标让模型更倾向 chosen，但 chosen 标注本身也可能有偏差。对齐不是事实验证器；仍需数据质量、离线评测、安全测试和线上监控。

10. 常见错误

- base 参数仍在优化器中：统计 `requires_ grad` ，并检查优化器参数组。
- 初始输出不同：检查 B 是否零初始化、缩放和 dropout。
- 训练后 base 改变：可能没有冻结，或错误地把合并权重写回后继续比较。
- SFT loss 包含 prompt：检查 labels 的 `-100` 区域。
- SFT label 边界错一位：先构造完整序列，再同时检查 `input_ ids= sequence[: -1]` 和右移后的 labels。

- padding 参与 loss：batch 中所有补齐位置的 label 也必须是 `-100` 。
- 把 LoRA 文件当完整模型：适配器通常依赖正确版本的基础模型。
- 先做 Full SFT 再比较 LoRA：两个分支起点不同，参数效率与原语料保持度对比失去意义。
- 用指令数据重建 Tokenizer：相同 token id 可能改变含义；必须使用 base checkpoint 内的冻结状态。
- 使用版本 1 的旧 base：请重跑实验 06→10；新评测不要求修改 base，但字符覆盖与上下文上限仍须满足。
- 用 dev 之外的数据调参：test 只在训练完成后评一次，不能反复查看后再选择配置。
- 只看训练 loss：同时检查 dev/test 任务指标、原语料验证 loss、冻结参数和 adapter 往返。

11. 出门测

1. `d_in=d_out=4096,r=8` 时，完整权重与 LoRA 可训练参数各是多少?
2. 为什么 LoRA 可以减少训练成本，却不保证推理模型本体很小?
3. SFT 和 LoRA 能否同时使用? 分别描述数据目标与参数集合。
4. 为什么低秩适配成功不等于解决灾难性遗忘、事实错误或安全问题?
5. 为什么 Full SFT 与 LoRA-SFT 必须各自从同一个 base checkpoint 分叉?

版本 2：失败记录和扩展评测

训练过程逐步保存历史，各分支及 seed 单独保存产物。`status` 区分 running/completed/failed/interrupted；`quality_status` 与逐项 `checks` 记录下降幅度、阈值、实际值和原因。效果不佳不再触发中途退出，所有 seed 继续完成；`--strict-checks` 在写完记录后返回 2。模型执行错误和冻结约束破坏属于失败，返回 1；用户中断返回 130。部分分支保留历史，但不会进入已完成分支的均值统计。`completed_seeds` 给出完整完成的 seed，单个指标的 `runs` 给出实际分母。

默认演示集仍为 4/4/4。`--eval-suite extended` 使用原有 4 条训练样本，新增 20 条 dev 和 40 条 test；测试专属意图不进入 dev。数据采用现有字符词表，模型知识没有因扩展评测而增加。每次报告分别记录数据 SHA-256 和评分实现 SHA-256。

CODE • POWERSHELL

```
.\.venv\Scripts\python.exe .\experiments\10_sft_tiny_gpt.py --quick --eval-suite
    extended --seed 42
```

每条样本的 `required_concepts` 是概念组列表：组内表达任选一个，组间必须全部满足；未声明时沿用原关键词。`contradictions` 声明已知矛盾表达。任务成功要求概念完整、未命中矛盾、格式正确；原始关键词准确率仍独立报告。逐样本结果列出 `missing_concepts` 、`matched_contradictions` 、`format_failures` 与生成结束原因 `stop_reason` （eos/length）。

例如“验证集损失帮助发现过度拟合。”可通过同义规则，而“验证损失不能帮助发现过拟合。”会命中否定反例。规则只覆盖已声明的表达，不能识别任意语义矛盾，不能当作人工理解的替代。固定评分器反例位于 `data/evaluation_cases.jsonl` 。

每次运行自动生成 PNG/SVG 原始 loss 曲线、可训练参数量与训练耗时图。耗时包含训练循环和记录开销，不能当作纯算子基准；不同 seed 汇总为均值与总体标准差。重新绘图无需模型：

CODE • POWERSHELL

```
.\.venv\Scripts\python.exe .\scripts\plot_experiments.py $SftRun --output-dir .\
    outputs\plots\lesson
```

第四部分

应用系统

评测、RAG 与 Agent：从会生成文本到可验证的系统

07 评测、RAG 与 Agent

对应 PDF：物理页 143-163。本章从“会输出文本”推进到可测量、可追溯、有失败边界的系统。

1. 先分清评测对象

对象	典型指标	不能单独回答的问题
语言模型	NLL、困惑度、任务准确率	应用是否引用正确、是否及时
检索器	Recall@k、MRR、nDCG	生成器是否忠实使用证据
RAG 端到端	答案正确率、忠实度、引用正确率	单凭总分难定位哪一层失败
Agent	任务成功率、工具错误率、步骤/成本/延迟	一次漂亮演示是否稳定

评测设计要先明确对象，否则“答案错了”无法判断是模型、检索、数据还是控制代码的问题。

2. 困惑度

平均负对数似然为 `NLL` 时：

$$PPL=\exp(NLL)$$

直觉上，它表示模型在每一步像是在多少个等概率候选中犹豫。注意：不同 Tokenizer 会改变序列长度和词表，困惑度不能不加说明地横向比较。

3. RAG 的完整数据流

CODE · TEXT

```
离线：文档 -> 切块 -> 向量化 -> 索引
在线：问题 -> 向量化 -> top-k 检索
                     -> 上下文 + 问题 -> 生成
                     -> 答案 + 来源
```

RAG 通常不更新模型参数。它把外部资料放入上下文，改善可追溯性和知识更新速度，但不会自动消灭幻觉。

本项目为了让每条断言都能由程序核对，使用“抽取式回答层”代替在线生成模型：它从最高相关文档复制句子并附上来源。这个基线展示完整的检索、拒答、引用和评测边界，但不代表真正的 LLM 生成能力。

4. Tiny RAG 实验

运行：

CODE · POWERSHELL

```
.\.venv\Scripts\python.exe .\experiments\09_tiny_rag.py
```

实验 09 和第 9 节的实验 11 全文，和仓库同一个提交：

 09_tiny_rag.py11_local_agent.py <noscript><p class="nb-code-status">看代码需要

JavaScript。源文件在 GitHub：experiments/09_tiny_rag.py、experiments/11_local_agent.py。</p></noscript>

实验只用仓库内 `data/ rag_ documents. json` ，流程是：

1. 读取文档和稳定 ID。
2. 英文转小写 token；中文使用字符 unigram 与 bigram。
3. 计算 TF-IDF 和余弦相似度。
4. 用余弦相似度检索 top-2。
5. 相关度低于阈值时明确回答“资料不足”，不生成虚假来源。
6. 相关度足够时，从证据中抽取句子并附加 `[document-id]` 。
7. 用严格的规范化包含关系检查每条回答是否受引用文档支持。
8. 在 6 个可回答问题和 2 个资料外问题上分别评测检索、拒答、支持性与端到端成功。

余弦相似度：

$$\cos(a,b)=(a\cdot b)/(||a||||b||)$$

预期四项指标都是 100%，并逐题打印检索分数、答案、来源和支持率。这里的数据集很小且人工设计，满分只证明教学管线满足这些样例，不代表生产质量。

5. Recall@k

若每个查询只有一个标注相关文档：

$$Recall@k=(正确文档进入前\ k\ 的查询数)/(查询总数)$$

k 越大，Recall 往往越高，但交给模型的无关上下文也可能更多、成本更大。不要只把 k 调到文档总数来“刷满分”。

6. RAG 失败矩阵

检索	生成	诊断
对	对	正常成功
对	错	生成未忠实使用证据、提示或模型问题
错	对	可能靠参数记忆猜中，不能算有证据成功
错	错	先改索引、切块、查询和检索器

必须保存 top-k 片段与分数，才能做这个分类。

7. 文档切块的取舍

- 太大：一个向量混合多个主题，检索不精确，输入 token 多。
- 太小：语义和上下文断裂，答案可能缺条件。
- overlap：减少跨边界信息丢失，但增加索引量和重复召回。

练习：把一个长段落分别按 50、100、200 字切块，标注 5 个查询，比较 Recall@k 和上下文长度。

8. “有来源”仍不等于“来源支持答案”

至少检查：

1. 引用 ID 是否真实存在。
2. 引用片段是否包含回答所需事实。
3. 回答有没有加入片段未支持的断言。
4. 多个来源冲突时是否表达不确定性。
5. 资料不足时是否能回答“不知道”。

9. Agent 是系统，不是单个提示词

最小工具循环：

CODE • TEXT

```
用户目标
  -> 模型提出结构化工具调用
  -> 程序校验工具名和参数
  -> 在权限边界内执行
  -> 结果写回状态
  -> 模型决定继续或给最终答案
```

必要组成：模型、工具、状态、控制循环。生产系统还需要权限、超时、幂等、审计、预算、终止条件和人工确认。

最小 Agent 练习

运行仓库中的确定性基线：

CODE • POWERSHELL

```
.\.venv\Scripts\python.exe .\experiments\11_local_agent.py
```

不接在线模型也能完整设计状态机：

- 规则规划器把任务路由到 `knowledge_ search(query)` 或 `calculator(expression)` 。
- 计算器用 Python AST 白名单解释基础算术，不使用 `eval` 或 shell。
- 控制器设置最大步数；每一步都保存 action、observation 和错误。
- 工具名必须来自白名单。
- 未知工具、工具异常和达到最大步数都有不同的停止状态。

实验必须同时通过正常算术、带引用检索、资料外拒答、未知工具拦截、除零错误和无限规划截断。规则规划器只是为了看清控制循环；换成 LLM 后，工具校验、权限和终止逻辑仍应由确定性程序负责。

10. 消融实验

至少做一个：

- 关闭检索，只给问题，比较事实回答。
- top-k 从 1 改 2/4。
- 去掉 ID，观察引用是否更容易编造。
- 去掉“资料不足则不知道”，记录无证据断言。
- 替换一个冲突文档，观察系统是否发现矛盾。

一次只改变一个设置，并保存相同查询集上的结果。

11. 出门测

1. Recall@5 很高但答案忠实度低，下一步应检查哪一层？
2. 为什么 RAG 不等于把知识永久写进模型？
3. 一个 Agent 可以执行删除操作时，哪些步骤必须由确定性程序而不是模型保证？
4. 设计一个“答案恰好正确但证据错误”的反例，并说明为何不能算 RAG 成功。
5. 为什么“模型承诺最多调用三次工具”不能替代控制器的 `max_ steps` ？

第五部分

独立实践

用结课项目证明你能提出问题、控制变量、复现实验并解释边界

08 结课项目：可复现的微型 LLM 实验报告

结课目标不是生成最像人类的句子，而是证明你能提出问题、实现机制、建立基线、控制变量、解释结果并诚实记录局限。

任选一个方向

A. TinyGPT 训练研究

研究问题示例：上下文长度、层数或 embedding 维度如何影响参数量、训练速度和验证 loss？

最低交付：两个配置、相同语料/seed/步数、训练曲线、生成样例和过拟合分析。

B. Attention 消融

研究问题示例：移除 `sqrt(d_ k)` 、移除 causal mask 或改变头数会发生什么？

最低交付：正确实现、一个错误/对照实现、shape 测试、未来泄漏测试、权重分布统计。

警告：移除 causal mask 的模型可能得到更低训练 loss，因为它偷看答案；这不是更好的语言模型。

C. LoRA 参数效率

研究问题示例：rank 1/2/4/8 对可训练参数、收敛速度和最终误差的影响。

最低交付：base 冻结证明、参数表、loss 曲线、合并输出检查。

D. Tiny RAG 评测

研究问题示例：chunk size、overlap 或 top-k 对 Recall 与上下文噪声的影响。

最低交付：不少于 10 篇小文档、20 个带标注查询、Recall@k、失败分类、来源示例和一次消融。

必须提交的目录

CODE · TEXT

```
report/
├── README.md              # 研究问题与结论
├── config/                # 所有配置和随机种子
├── results.csv            # 原始指标，不只截图
├── examples.md            # 成功与失败样例
└── limitations.md         # 局限、风险和下一步
```

代码应能从项目根目录执行，不要硬编码本机绝对路径。

仓库提供了一份可重新运行的完整示例：Attention 缩放消融报告。先运行它，核对 @@INLINE0000@@ 与结论的对应关系，再选择自己的问题；不要只改标题或复制示例结论。示例的脚本全文，和仓库同一个提交：

 run.py <noscript> 看代码需要 JavaScript。源文件在 GitHub：examples/capstone_attention/run.py。 </noscript>

报告结构

1. 问题：可证伪、只包含主要自变量。
2. 假设：运行前预期及理由。
3. 方法：数据、切分、模型、参数、seed、硬件。
4. 基线：随机、未训练、关闭某组件等。
5. 指标：为何这个指标能回答问题。
6. 结果：原始数据和汇总，不隐藏失败 run。
7. 分析：现象是否支持假设，有哪些替代解释。
8. 失败案例：至少 3 个，按根因分类。
9. 局限：哪些结论不能外推。
10. 复现命令：新终端里从环境检查开始。

最低复现命令

CODE - POWERSHELL

```
.\.venv\Scripts\python.exe .\experiments\00_environment_check.py
.\.venv\Scripts\python.exe -m unittest discover -s .\tests -v
.\.venv\Scripts\python.exe 你的实验脚本.py
```

评分量表（100 分）

维度	分值	满分标准
环境与复现	20	venv、依赖、seed、命令和数据路径完整
实现正确性	25	单测覆盖关键不变量；无数据泄漏
实验设计	20	有基线，单变量，有合理指标
结果与分析	20	原始数据、趋势、失败案例、替代解释
文档与边界	15	能清晰复述，并诚实说明玩具实验局限

完成定义

- 一条命令可运行快速测试。
- 另一条命令可重现实验主要结果。
- 至少一个结论由自动检查或量化指标支持。
- 至少一个原假设被数据修正，或说明为何没有。
- 明确区分“代码管线运行成功”和“模型能力足够好”。

可复制的版本 2 配置消融

以下示例只改变 embedding 维度，使用相同 seed、步数、语料和其他配置。输出目录带时间标签，重复运行不会覆盖旧实验。

CODE · POWERSHELL

```
$Tag = Get-Date -Format yyyyMMdd-HHmmssfff
$RunA = ".\outputs\pretrain\width32-$Tag"
$RunB = ".\outputs\pretrain\width64-$Tag"
.\.venv\Scripts\python.exe .\experiments\06_train_tiny_gpt.py `
  --quick --seed 42 --n-embd 32 --output-dir $RunA
.\.venv\Scripts\python.exe .\experiments\06_train_tiny_gpt.py `
  --quick --seed 42 --n-embd 64 --output-dir $RunB
.\.venv\Scripts\python.exe .\scripts\plot_experiments.py `
  $RunA $RunB --output-dir ".\outputs\plots\width-$Tag"
```

每个运行目录有 `pretraining. json` 、 `pretraining. csv` 和 `plots/` 。比较图显示原始 loss 曲线、可训练参数量和耗时， `runs. json` 将图中短标签对应到完整配置与数据指纹。英文图内标签避免中文字体依赖：Training/Validation loss 表示训练/验证损失，Trainable parameters 是可训练参数量，Training duration 是包含记录开销的训练循环耗时。

默认不会因为效果不达标丢掉实验。报告必须同时检查执行 `status` 、效果 `quality_ status` 、逐项 `checks` 与完成的 seed 数量；不要将缺失 seed 当零。跨配置保持各组独立，均值和总体标准差只在同一运行的同一分支内计算；单 seed 的标准差为零不代表估计没有不确定性。

要在其中一份 base 上做 SFT，显式传 `--base-checkpoint "$RunA\ base. pt"` 。SFT 结束后通过 `--run-dir` 、 `--branch full|random|lora` 和 `--seed` 指定具体模型。旧版产物不兼容版本 2；重新训练时保留旧文件用于核对，不混合新旧指标。

附录

模板、参考与勘误

实验记录模板 · 《Happy-LLM v1.0》物理页码对照与代码风险

实验记录模板

基本信息

- 日期：
- 实验名称：
- Git/文件版本（若有）：
- Python / PyTorch：
- 设备：CPU / GPU：
- 随机种子：
- 运行命令：

1. 问题

今天要解释或检验什么？写成一个可以被结果否定的问题。

2. 运行前预测

- 输入 shape：
- 输出 shape：
- 预期数值关系/趋势：
- 理由：

3. 控制变量

- 本次只改变：
- 保持不变：
- 基线：

4. 结果

配置	指标 1	指标 2	耗时	备注
baseline				
experiment				

关键原始输出或结果文件路径：

5. 解释

- 结果是否支持预测？
- 哪个机制最可能造成现象？
- 还有哪些替代解释？

6. 失败与排查

症状/错误	假设原因	检查动作	根因	修复

7. 结论边界

- 这个实验能说明：
- 这个实验不能说明：
- 若要外推，需要补充：

8. 出门测

不用看代码，用自己的话解释核心机制；再写一道计算题和一个可能的 bug。

《Happy-LLM v1.0》章节对照、公式索引与勘误

本文中的“物理页”指 PDF 文件从 1 开始计数的实际页序号，不是正文中可能出现的章节页码。PDF 共 163 个物理页。

本项目是参考《Happy-LLM v1.0》知识脉络重新设计的原创教学改编。项目中的讲解、代码、实验、测试和验收标准均为重新组织与实现，不是对 PDF 正文或源码的逐字复制。PDF 页码仅用于回查主题；当 PDF 示例与本项目验证结果不一致时，以本项目中通过测试的实现和本文勘误为准。

1. 全书物理页对照

PDF 物理页	原书内容	在本课程中的作用
1-6	项目介绍、前言、学习建议	了解原书目标；本项目另行补足零基础数学、环境隔离和可验证实验方法
7-14	第 1 章：NLP 基础概念	建立 token、词表、语言模型和文本表示的直觉
15-31	第 2 章：Transformer 架构	课程 04-05 的主要理论来源：注意力、掩码、多头、归一化、残差和位置编码
32-57	第 3 章：预训练语言模型	比较 Encoder-only、Encoder-Decoder、Decoder-only 及 BERT、T5、GPT、LLaMA、GLM
58-73	第 4 章：大语言模型	理解 Pretrain、SFT、RLHF 的目标差异，以及数据、算力和对齐问题
74-116	第 5 章：动手搭建大模型	从 RMSNorm、GQA、RoPE、MLP 到 Tokenizer、Dataset、预训练和 SFT 的综合参考
117-142	第 6 章：大模型训练流程实践	Transformers、Trainer、DeepSpeed、Adapter、Prefix Tuning、LoRA
143-163	第 7 章：大模型应用	评测、RAG 和 Agent；适合作为完成核心实验后的扩展项目

1.1 与核心实验最相关的细分页码

PDF 物理页	主题	建议回查时关注什么
12-13	语言模型、Word2Vec	“根据上下文预测 token”和向量点积的最早直觉
15-18	Q、K、V 与缩放点积注意力	从相似度、Softmax 到 Value 加权和
18-19	自注意力与因果掩码	为什么未来位置必须被遮挡，为什么用 `-inf`
19-21	多头注意力	每个头的投影、并行计算、拼接和输出投影
21-25	Encoder-Decoder、FFN、LayerNorm、残差	Transformer Block 的组成部件
25-30	Embedding、位置编码、完整 Transformer	token id 如何进入网络，以及位置信息如何注入
58-61	LLM 的定义、能力和局限	上下文学习、指令遵循、推理、长文本、幻觉
62-65	Pretrain、并行训练、数据清洗	CLM 目标、数据/模型并行、过滤与去重
66-69	SFT	指令数据、多轮对话，以及只在 assistant 回复上计算损失
69-72	RLHF	Reward Model、PPO 的四模型结构、KL 约束和 DPO 思路
74-87	手写 LLaMA2	RMSNorm、GQA、RoPE、SwiGLU、DecoderLayer、采样
87-103	Tokenizer 与 Dataset	BPE、特殊 token、X/Y 右移、预训练和 SFT loss mask
103-112	预训练与 SFT 循环	学习率、梯度累积、裁剪、AMP、checkpoint
135-142	高效微调与 LoRA	低秩旁路、冻结基座、目标层、PEFT 配置
148-154	Tiny-RAG	切分、向量化、余弦相似度、top-k 检索和上下文生成

2. 课程实验相对前一阶段增加了什么

课程刻意把原书中的大模型拆成六个可以在本机验证的小台阶。每个实验只引入少量新概念，并保留上一实验已经验证过的能力。

实验	相对前一阶段新增的能力	主要不变量或验收点	PDF 物理页
@@INLINE0000@@	建立项目内 `. venv`，确认解释器、依赖、设备和随机种子可复现	Python 位于 `. venv`；NumPy/PyTorch 可导入；CPU 也通过；同种子结果一致	7-30 的实验前置

实验	相对前一阶段新增的能力	主要不变量或验收点	PDF 物理页
@@INLINE0000@@	从普通数值进入张量计算：点积、矩阵乘法、数值稳定 Softmax、交叉熵	shape 正确；Softmax 每行和为 1；极大 logits 不产生 `nan`；正确类别概率增大时 CE 下降	12-13、16-20
@@INLINE0000@@	把字符串变成 token/id，并能从 id 还原；显式处理词表和未知字符	编码结果为整数序列；已知文本往返一致；未知字符行为有定义	7-13；扩展阅读 87-93
@@INLINE0000@@	从静态 token 映射进入最小“预测下一个 token”模型；增加 NLL、PPL 与采样	概率归一；NLL/PPL 可解释；固定种子采样可复现；模型仅使用前一个 token	12-13
@@INLINE0000@@	用 Q、K、V 替代 bigram 查表；增加缩放点积注意力和因果掩码	权重和为 1；输出 shape 正确；被遮挡权重为 0；修改未来 token 不改变过去位置	15-20
@@INLINE0000@@	在单头注意力上增加多头拆分/合并、残差、归一化和 FFN，组成 Decoder Block	`D % H = = 0`；输入输出均为 `(B,T,D)`；因果性不被多头和 FFN 破坏；反向传播有有限梯度	19-30

这里选择 Decoder-only Block，是为了直接连接现代生成式 LLM。它吸收了原书第 2 章的基础组件，但没有照搬原书的完整机器翻译 Encoder-Decoder 示例。

3. 公式索引：保留原书符号，再说明课程中的含义

3.1 点积、Softmax 与注意力（物理页 17）

原书先用字典权重给出：

$$value=0.6\times10+0.4\times5+0\times2=8$$

向量点积写为：

$$v\cdot w=\Sigma_i v_iw_i$$

单个查询与全部 Key 的相似度：

$$x=qK\hat{}T$$

Softmax：

$$softmax(x)_i=\frac(e\hat{}(x_i))(\Sigma_j e\hat{}(x_j))$$

从单查询推广到矩阵，并加入缩放后，核心公式为：

$$\text{attention}(Q,K,V)=\text{softmax}\leqslant ft((QK\hat{}T)/(\sqrt{}(d_k)))V$$

课程 01 验证点积和稳定 Softmax，课程 04 才把它们组合成完整注意力。`\ sqrt{d_ k}` 用于控制点积随维度增大而增长，减轻 Softmax 过度饱和。

3.2 多头注意力（物理页 20）

$$\text{MultiHead}(Q,K,V)=\text{Concat}(\text{head}_1,\ldots,\text{head}_h)W\hat{}O$$

$$\text{head}_i=\text{Attention}(QW_i\hat{}Q,KW_i\hat{}K,VW_i\hat{}V)$$

课程 05 重点不是背公式，而是追踪形状：

CODE · TEXT

```
(B,T,D) -> (B,H,T,D_h) -> (B,H,T,T)
        -> (B,H,T,D_h) -> (B,T,D)
```

其中 `D_ h = D / H` 。

3.3 归一化（物理页 23）

原书先以批归一化的符号说明均值、方差和标准化：

$$\mu_j=(1)/(m)\Sigma_(i=1)\hat{}(m)Z_j\hat{}i$$

$$\sigma\hat{}2=(1)/(m)\Sigma_(i=1)\hat{}(m)(Z_j\hat{}i-\mu_j)\hat{}2$$

$$\widetilde{Z_j}=(Z_j-\mu_j)/(\sqrt{}(\sigma\hat{}2+\varepsilon))$$

LayerNorm 使用同类计算，但统计轴不是 batch 轴，而是每个 token 的特征轴。课程 05 应检查归一化后的均值和方差近似目标值，不应声称数据因此服从正态分布。

3.4 正余弦位置编码（物理页 26）

$$PE(pos,2i)=\sin\leqslant ft(pos/10000\hat{}(2i/d_(model)))$$

$$PE(pos,2i+1)=\cos\leqslant ft(pos/10000\hat{}(2i/d_(model)))$$

这是原始 Transformer 的绝对位置编码。课程 05 的最小 Decoder Block 可以使用更简单的位置处理，但阅读时要区分它与 LLaMA 使用的 RoPE。

3.5 训练计算量的历史近似（物理页 63）

原书列出：

$$C\tilde{}6ND$$

其中 `C` 为计算量、`N` 为参数量、`D` 为训练 token 数。它适合帮助理解参数、数据和算力之间的关系，不是任何时代、模型和硬件上都固定成立的配方。

3.6 RMSNorm（物理页 75）

$$RMSNorm(x)$$
$$=\frac{x}{\sqrt{\frac{1}{n}\Sigma_{i=1}\hat{}n x_i\hat{}2+\varepsilon}}\cdot\gamma$$

它不减均值，只按均方根缩放，再乘可学习参数 `\ gamma`。这是第 5 章 LLaMA 风格模型与第 2 章 LayerNorm 的重要区别。

3.7 LoRA（物理页 137-138）

原权重冻结，更新量写成低秩乘积：

$$W_0+\Delta W=W_0+BA,$$
$$B\in R\hat{}(d\times r),\ A\in R\hat{}(r\times k)$$

前向传播：

$$h=W_0x+\Delta Wx=W_0x+BAx$$

原书在方阵近似下给出的可训练参数量为：

$$\Theta=2\times L_{LoRA}\times d_{model}\times r$$

更一般地，一个 `d_ out x d_ in` 线性层增加 `r(d_ in+d_ out)` 个权重参数。`r` 越小，参数越少，但表达能力也可能受限。

3.8 RAG 的余弦相似度（物理页 150）

原书以代码实现点积除以两个向量的模，等价于：

$$\cos(a,b)=(a\cdot b)/(||a||||b||)$$

余弦相似度只衡量向量方向接近程度。RAG 是否给出可靠答案，还取决于切分、Embedding、top-k、提示模板和生成模型，不能只看这个分数。

4. 不要盲目照抄：原书示例勘误与课程处理

PDF 物理页	原书示例的风险	本项目中的处理
17	“语义相似的向量点积应大于 0、不相似应小于 0”不是一般数学保证；点积还受向量长度和训练方式影响	只把点积解释为当前表示空间中的相似度打分；课程 01 同时观察数值、方向和 Softmax 结果
18	`attention(x,x,x)` 只表达 Q/K/V 来自同一序列，省略了可学习的 `W_ Q/ W_ K/ W_ V`	课程 04 先讲最小核心；课程 05 显式加入独立投影层
23	“归一化后变成标准正态分布”过强；零均值、单位方差不等于服从正态分布	验收均值、方差和有限数值，不作分布形状保证
24	`EncoderLayer` 示例先用归一化结果覆盖 `x`，随后再将该值当残差，容易丢失真正的原始残差分支	保存原输入；采用清晰的 Pre-Norm 结构 `x = x + sublayer(norm(x))`
29-30	示例把同一 `x` 同时作为 Encoder 和 Decoder 输入，没有展示右移后的目标序列；`get_ num_ params(non_ embedding= True)` 还访问不存在的 `wpe. weight`	核心课程不把该片段当作可训练机器翻译实现；课程 05 构造经过单测的 Decoder-only Block
75	配置示例默认 `dim= 768`，RMSNorm 测试输出却写成末维 `288`	输出末维必须等于实际输入 `D`；测试直接断言 `output. shape = = input. shape`
92、99	文中称为正确输出，但实际显示 `Decoded text matches original: False`、`Special tokens preserved: False`	课程 02 对已知字符要求严格往返；特殊 token 与空格策略必须通过显式测试
100	配置把 `<|im_ end|>` 设为 pad token，Dataset 却硬编码 `padding= 0`，并依赖 `ignore_ index= 0`；语义不一致	始终读取 `tokenizer. pad_ token_ id`，并用独立 loss mask 或统一的 `ignore_ index`
101	assistant 起始序列被硬编码为 `[3,1074,537,500,203]`，更换 tokenizer 后会失效	由当前 tokenizer 动态编码 assistant 边界，或在结构化消息转换时直接生成 mask
103 以后	默认约 215M 模型、多 GPU、DataParallel、CUDA AMP、SwanLab/API key，不适合作为零基础本地首跑；整份大语料 `readlines()` 也可能耗尽内存	核心实验默认 CPU、小张量、小语料、无外部登录；先做单 batch 过拟合，再按硬件逐步放大；大数据使用流式读取
154	`VectorStore. query` 的定义需要 `EmbeddingModel`，Demo 却传 `model= 'zhipu'`；本地模型还硬编码 `. cuda()`	接口统一传入 Embedding 实例；设备通过参数选择，CPU 路径必须可运行
多处	小节编号错位或重复，例如物理页 87 的 `5. 3. 1`、物理页 112 重复 `5. 3. 4`，部分标题存在缺字	以本文物理页和主题名定位，不依赖原书小节编号的连续性

5. 如何使用这份对照

1. 先运行当前编号的课程实验，再带着输出和问题回查 PDF 对应物理页。
2. 阅读公式时，在纸上标出每个张量的 shape；无法写出 shape 时，不进入下一公式。
3. PDF 中的大模型代码先看结构，不直接运行默认配置；先在课程的小模型上验证同一原理。

4. 遇到本文勘误中的片段，先解释为什么存在风险，再对比本项目实现和测试。

5. 完成课程 05 后，再按个人硬件选择第 5-7 章的 Tokenizer、Tiny-LLaMA、LoRA、RAG 或 Agent 扩展；这些扩展不是核心六实验的前置条件。

学习本项目的目标不是复述书中的代码，而是能够从公式推出 shape，从 shape 写出实现，再用测试证明实现符合因果性、概率归一和数值稳定等不变量。

- 全书完 -

源文件：LearnLLM 工作区 Markdown 讲义

生成脚本：scripts/build_learning_guide_pdf.py